



INSTITUTE OF INFORMATION TECHNOLOGY
UNIVERSITY OF DHAKA



TERRA-AUDIT

– AI-Assisted Digital MRV Platform for AWD Rice Irrigation and Agricultural Land Management Carbon Credits

Submitted As

Technical Report

SE801 Project Final Technical Report

Submitted By

Kazi Nahid

BSSE Roll: 1437

Supervised By

Dr. Emon Kumar Dey

Professor

Institute of Information Technology
University of Dhaka

Submission Date

Oct 6, 2026

Letter of Transmittal

October 6, 2026

BSSE 4th Year Exam Committee

Institute of Information Technology

University of Dhaka

Subject: Final Report Submission — Terra-Audit: AI-Assisted Digital MRV Platform for AWD Rice Irrigation and Agricultural Land Management Carbon Credits

Dear Sir,

I am submitting the technical report for my Software Project Lab 3 project, Terra-Audit: AI-Assisted Digital MRV Platform for AWD Rice Irrigation and Agricultural Land Management Carbon Credits. This report covers the project's requirements, software design, implementation, interface design, testing, user guidance, and the limitations of the current system. I welcome your feedback and suggestions for further improvement.

Thank you for your consideration.

Sincerely,

Kazi Nahid

BSSE-1437

Institute of Information Technology

University of Dhaka

October 8, 2026

Supervisor's Signature

Abstract

Terra-Audit is an evidence-centered digital monitoring, reporting, and verification (MRV) platform for agricultural carbon-project preparation. The final implementation supports two deliberately scoped accounting pathways: VM0051 v1.1 for Alternate Wetting and Drying (AWD) rice and VM0042 v2.2 for agricultural land management (ALM), with a restricted VMD0054 v1.1 leakage adapter. The system uses a Next.js web frontend and FastAPI backend with organization/project access control, structured evidence records, methodology bundles and readiness checks, immutable calculation snapshots, durable background jobs, internal review, and audit-oriented exports. Sentinel-1 analytics, Random Forest/XGBoost experiments, managed crop-model workflows, and bounded generative-AI explanations remain supporting or experimental features; deterministic server code retains authority over accounting equations, readiness states, and stored numerical results.

Recorded final verification reports 177 backend/core tests passed, with the recorded frontend tests, production build, TypeScript check, and lint also completing successfully. Methodology coverage remains incomplete; common.additionality is unsupported and blocking; independent AWD field ground truth and the canonical generative-AI evaluation set are incomplete; and production worker, database/storage, and deployed-revision provenance have not been verified end to end. Software verification, internal review, external VVB verification, and Verra registration or issuance are therefore treated as distinct assurance levels throughout this report.

Keywords: digital MRV, AWD rice, VM0051, VM0042, carbon accounting, evidence provenance, satellite monitoring, readiness, internal review.

Table of Contents

1. Project Overview.....	1
1.1 Project Title.....	1
1.2 Problem Statement.....	1
1.3 Objectives.....	1
1.4 Scope.....	2
1.5 Deliverables.....	3
2. Requirements Analysis.....	4
2.1 Requirements Inception.....	4
2.2 Functional Requirements.....	4
2.3 Non-functional Requirements.....	5
2.4 Stakeholders and Viewpoints.....	7
2.5 Quality Function Deployment and Prioritization.....	8
2.6 Representative User Stories.....	8
3. System Modeling.....	8
3.1 Use Case Diagram and Descriptions.....	8
3.1.1 Use Case Descriptions.....	11
3.2 Activity Diagram.....	12
3.2.1 Cropland ALM Detailed Activity Flow.....	14
4. Data & Information Modeling.....	14
4.1 Entity-Relationship Model.....	14
4.1.1 High-Level Entity Relationship Diagram.....	15
4.1.2 Focused Relationship Summary.....	16
4.2 Database Schema.....	16
4.3 Dataset Description (AI-Assisted Detection).....	17
5. Software Design.....	18
5.1 Component Architecture.....	18
5.2 Design Classes and Responsibility Boundaries.....	19
5.2.1 CRC Model.....	19
5.2.2 Implementation Class and Method Mapping.....	21
5.3 Persistent Data Sources and Interfaces.....	22
5.3.1 Context-Level Data Flow (DFD Level 0).....	22
5.3.2 Level-1 Data Flow (DFD Level 1).....	23
5.4 Behavioral Design.....	23
5.4.1 Behavioral Event Table.....	24
5.4.2 Sequence Diagrams.....	25
5.5 Deployment Design.....	25
5.6 Design Evolution and Alternatives.....	26
6. AI Engineering Design.....	26
6.1 Classical AWD ML Pipeline.....	26
6.2 Model Selection and Evaluation.....	27
6.3 Managed Crop Models.....	27
6.4 Generative AI Explanation Architecture.....	27
6.5 Prompt and Context Design.....	28
6.6 AI Authority Boundary and Evaluation Limitations.....	28
7. Testing Strategy and Results.....	29
7.1 Testing Objectives.....	29
7.2 Test Strategy and Pass/Fail Criteria.....	29
7.2.1 Representative Direct API Endpoint Verification.....	29
7.2.2 Selected Detailed Test Outcomes.....	31
7.3 Final Verification Results.....	33
7.4 Risks and Contingencies.....	35
7.5 Requirements Traceability Matrix.....	36
8. Final Implementation.....	37
8.1 Final Feature Status.....	37
8.2 Remaining Engineering and Validation Gaps.....	39
9. Project Timeline and Final Status.....	40
9.1 Planned Schedule and Final Verification Point.....	40
10. Interface Design.....	42
10.1 Interaction Objects and States.....	42
10.2 Primary User Workflows.....	42
10.2.1 UI Event-to-State Transition Summary.....	42
10.3 Responsive, Validation and Accessibility Design.....	43
10.4 Interface Evidence Boundary.....	43
11. User Manual, Deployment and Repository.....	47
11.1 Sign In and Select Project Context.....	47
11.2 Register a Field and Build Evidence Context.....	47
11.3 Rice Signal Analytics and Calculation.....	48
11.4 ALM Evidence and VM0042 Calculation.....	48
11.5 Methodology Readiness and Internal Review.....	48
11.6 AI Explanation.....	49
11.7 Export and Troubleshooting.....	49
11.8 Local Installation and Development Setup.....	49
11.9 Deployment and Public Repository.....	50
12. Evaluation, Limitations and Conclusion.....	50
12.1 Final Evaluation.....	51
12.2 Methodology and Accounting Limitations.....	51
12.3 Scientific Validation Limitations.....	51
12.4 Provenance and Reproducibility Limitations.....	52
12.5 Security and Operational Limitations.....	52
12.6 Generative-AI Limitations.....	52
12.7 Prioritized Future Work.....	52
12.8 Conclusion.....	52
Appendix A — Selected Technical Persistence Diagram.....	53
References.....	54

List of Figures and Tables

Figures

- Fig 1 Use-case context — p. 10
- Fig 2 Detailed use-case model — p. 10
- Fig 3 Carbon Calculation use case — p. 11
- Fig 4 Overall activity flow — p. 13
- Fig 5 Versioned Carbon Calculation activity — p. 13
- Fig 6 Methodology Readiness and Internal Review activity — p. 14
- Fig 7 High-level entity-relationship model — p. 15
- Fig 8 Component architecture — p. 18
- Fig 8A CRC model for core design classes — p. 20
- Fig 8B Context-level data flow (DFD Level 0) — p. 23
- Fig 8C Level-1 data flow (DFD Level 1) — p. 23
- Fig 9(a) Durable jobs and calculation state transitions — p. 24
- Fig 9(b) Internal review and data-quality state transitions — p. 25
- Fig 9(c) Internal review sequence — p. 25
- Fig 9(d) Calculation commit sequence — p. 26
- Fig 10 Supported deployment topology — p. 27
- Fig 11 AI authority and data flow — p. 29
- Fig 12 Project timeline — p. 41
- Fig 13 Application shell and project navigation — p. 44
- Fig 14 Monitoring batch state — p. 45
- Fig 15 Rice calculation readiness and inputs — p. 46
- Fig 16 Review-related interface states — p. 47
- Fig 17 AI workspace - crop-model controls — p. 48
- Fig A1(a) Calculation and internal-review persistence — p. 54

Tables

- Table 1 Functional Requirements and Priorities — p. 4
- Table 2 Non-functional Requirements and Verification Targets — p. 5
- Table 3 Stakeholders and Viewpoints — p. 7
- Table 4 Use-Case Inventory and Actor Mapping — p. 9
- Table 5 Detailed Use-Case Specifications — p. 11
- Table 6 Selected Persistent Database Schema — p. 16
- Table 7 Component Responsibilities — p. 18
- Table 7A Behavioral event table — p. 24
- Table 8 Recorded Final Verification Outcomes — p. 34
- Table 9 Functional Requirements Traceability Matrix — p. 37
- Table 10 Final Implementation by Subsystem — p. 38
- Table 11 Remaining Engineering and Validation Gaps — p. 40
- Table 12 Project Phases, Planned Windows, and Final Recorded Status — p. 41

1. Project Overview

1.1 Project Title

Terra-Audit — AI-Assisted Digital MRV Platform for AWD Rice Irrigation and Agricultural Land Management Carbon Credits.

This final report consolidates the project's earlier requirements analysis, SRS-style modeling, midterm technical report, final software design, implementation, interface design, testing evidence, and user guidance. Earlier documents are retained as development history, while descriptions of the implemented system in this report are aligned with the final implemented web system rather than with the retired Streamlit-era prototype.

1.2 Problem Statement

Rice paddies are a major source of anthropogenic methane, and Alternate Wetting and Drying (AWD) irrigation is a recognized mitigation practice, but evidence collection and carbon accounting remain difficult to scale and audit. Farmer irrigation logs can be incomplete or difficult to verify, continuous field instrumentation can be costly across many smallholder fields, and spreadsheet-based accounting often separates the reported result from the evidence, assumptions, methodology version, and calculation history that produced it. A parallel problem exists for improved agricultural land management (ALM), where practice changes, production data, soil organic carbon (SOC) measurements, leakage considerations, and uncertainty must be assembled from different evidence sources and retained consistently across verification periods.

Terra-Audit addresses these problems as an evidence-first digital MRV platform with two scoped accounting pathways. For rice fields, Sentinel-1 SAR observations from Google Earth Engine support analysis of flooding and drydown behavior; for cropland ALM fields, structured practice, soil, laboratory, production, livestock, and leakage evidence supports the accounting workflow. The platform combines deterministic carbon calculations, methodology-bundle and readiness checks, versioned calculation snapshots, internal review, durable background processing, and exportable evidence records. Satellite and AI outputs are treated as supporting analytical evidence rather than as substitutes for independent field validation or external VVB/Verra verification.

1.3 Objectives

- Use freely available Sentinel-1 SAR observations to provide scalable supporting evidence of rice-field flooding and drydown behavior while clearly separating satellite-derived analytics from independently validated AWD ground truth.
- Provide a parallel evidence-driven cropland ALM pathway based on the project's scoped implementation of Verra VM0042 v2.2, using structured practice, production, livestock, soil-sampling, laboratory, custody, and reviewed SOC evidence rather than rice satellite signals.
- Implement transparent deterministic carbon accounting for the supported subsets of Verra VM0051 v1.1 (rice AWD, QA3/default-factor pathway) and VM0042 v2.2 (cropland ALM), exposing important intermediate values and methodology assumptions for review.
- Represent the evidence lifecycle explicitly, including crop seasons and corrections, observations, practice events, attachments, production records, soil sampling, laboratory

results, chain-of-custody records, reviewed SOC inputs, and supported integrated crop-livestock evidence.

- Integrate methodology bundles, corrections, applicability, readiness checks, and restricted VMD0054 v1.1 leakage logic so unsupported, incomplete, or reviewer-dependent requirements are disclosed instead of being silently treated as satisfied or zero.
- Preserve calculation history through versioned calculation records and frozen evidence/input/result snapshots, and support internal review of eligible calculation versions without presenting that workflow as external VVB validation or registry verification.
- Support AI-assisted analysis while keeping deterministic accounting authoritative: classical Random Forest/XGBoost models and managed crop models remain experimental, while generative AI is limited to bundle-scoped draft explanations that are validated by server-side rules and require human review.
- Provide an authenticated multi-user web workflow with organization/project access control, durable background jobs, calculation history, review and export facilities, and sufficient traceability for audit preparation while clearly disclosing current security, provenance, and methodology limitations.

1.4 Scope

In scope:

- Authenticated account, organization, project, farm, and field workflows, including field registration from supported geometry inputs and server-side organization scoping.
- Two registered accounting pathways — `rice_awd` and `cropland_alm_vm0042` — with field type treated as an immutable routing attribute after registration.
- Rice signal analytics using Sentinel-1 VV/VH observations, compatible caching, durable signal jobs, smoothing, threshold-based AWD/phenology logic, and optional experimental ML detectors.
- Structured cropland evidence for VM0042, including crop seasons, practice schedules, production records, soil sampling plans/strata/samples, laboratory results, custody history, reviewed SOC mapping, and supported non-SOC emission inputs.
- Baseline/project livestock schedules for the supported integrated crop-livestock scope.
- Supported livestock emission terms, including selected enteric-fermentation and pasture/range/paddock manure sources within the scoped ALM engine.
- Project-, field-, bundle-, and reporting-period-scoped leakage assessment using the implemented restricted VMD0054 v1.1 adapter and structured production evidence.
- Versioned calculation history with frozen inputs, results, readiness state, methodology-bundle context, and supersession metadata; eligible versions can enter the internal review workflow.
- Methodology document/bundle management, corrections, project applicability, deterministic readiness evaluation, findings, and reviewer workflow.
- Deterministic scoped accounting for VM0051 v1.1 rice and VM0042 v2.2 ALM, including the implemented uncertainty, leakage, SOC, and selected non-SOC terms.
- Classical AWD ML, managed crop-model experimentation, and generative draft explanations, all explicitly separated from authoritative accounting and external verification decisions.

- Supported export paths include legacy verification PDF/JSON/CSV, versioned calculation JSON, and snapshot-backed ALM calculation PDF; the application also provides private evidence attachments, calculation history, and operational/admin views for jobs, methodology sources, providers, and worker state.
- Primary research focus remains Bangladesh/South Asia for the rice pathway. The application supports SQLite for local/test use and PostgreSQL plus local or private S3-compatible storage configurations for hosted operation.

Out of scope (this phase):

- Independent field-level AWD ground-truth validation and continuous on-field sensor integration are not completed in the current project; the satellite detector remains an experimental/supporting evidence source.
- Carbon-accounting pathways other than the implemented VM0051 and VM0042 scopes, and complete implementation of every requirement, tool, safeguard, and procedure associated with those methodologies and the VCS program.
- External validation/verification by a VVB, Verra registry integration, VCU creation/issuance, and any claim that an internally calculated quantity is an issued carbon credit.
- Complete production-security certification and operational assurance, including comprehensive project-level authorization coverage, database foreign-key/row-level-security enforcement, independent penetration testing, verified backup/restore drills, and fully sealed end-to-end provenance for every legacy export path.

1.5 Deliverables

- A Next.js web frontend and FastAPI backend providing authenticated project, field, evidence, monitoring, calculation, review, AI, export, and administration workflows; the legacy Streamlit runtime has been retired.
- A domain-oriented persistence layer supporting SQLite and PostgreSQL, with structured records for accounts, projects, fields, evidence, soil, production, calculations, reviews, methodologies, jobs, AI records, and private attachment/model storage.
- Deterministic scoped accounting engines for VM0051 v1.1 rice and VM0042 v2.2 ALM, including restricted VMD0054 v1.1 leakage integration, methodology/readiness gates, and versioned calculation persistence.
- A durable worker/job system for signal analytics and AI work, together with the Sentinel-1 threshold pipeline, experimental RF/XGBoost and managed crop models, and provider-backed generative explanation infrastructure.
- Structured evidence management, private attachments, calculation snapshots/history, internal review/findings, and workflow-specific export paths: legacy verification PDF/JSON/CSV, versioned calculation JSON, and snapshot-backed ALM calculation PDF, with legacy-provenance limitations disclosed.
- Methodology source/bundle management, project applicability and readiness checks, calculation history/portfolio views, operational monitoring, and server-validated draft AI explanations.
- Final technical documentation, user guidance, a public version-controlled repository, and recorded local verification results: 177 backend/core tests reported as passing with no failures or skips; frontend tests, production build, and TypeScript checks were also

recorded as passing, while ESLint recorded five warnings. These commands were not rerun during preparation of this final report.

2. Requirements Analysis

2.1 Requirements Inception

The original SRS established Terra-Audit around the need to make agricultural MRV evidence more traceable and less dependent on disconnected manual records. During implementation, that problem expanded from a single-user rice prototype into a multi-user platform covering project/field management, structured evidence, two scoped carbon-accounting pathways, methodology readiness, durable processing, internal review, and AI-assisted analysis. The final requirements therefore retain the original evidence-first motivation while reflecting the capabilities and constraints of the final implementation.

Requirement elicitation combines the earlier SRS and midterm analysis with implementation evidence from the final system. Carbon-project users need transparent calculations and evidence history; researchers need reproducible satellite/ML workflows; reviewers need explicit readiness and provenance; administrators need secure, observable operations; and external VVB/Verra processes remain outside the application authority boundary. Requirements that cannot be fully automated are represented as partial, unsupported, blocking, or human-review dependent rather than being treated as silently complete.

2.2 Functional Requirements

Table 1: Functional Requirements and Priorities

ID	Requirement	Priority
FR1	Provide authenticated account access and account-recovery flows; derive the active organization from the authenticated user rather than from a client-selected tenant identifier.	Must
FR2	Create and manage projects, farms, and fields; register each field with a supported geometry and an immutable accounting pathway (rice_awd or cropland_alm_vm0042), and maintain dated project/farm memberships.	Must
FR3	For rice fields, request Sentinel-1 VV/VH observations for a monitoring window, reuse compatible cached or active work where possible, and otherwise execute acquisition/processing through the durable signal-job workflow.	Must
FR4	Process rice time-series with the implemented smoothing, flood/drydown and phenology logic, and retain threshold-based detection as the deterministic reference path for experimental AWD ML models.	Must
FR5	Record crop seasons with correction/version history and associate field observations, practice events, monitoring runs, reviews, data-quality issues, and evidence attachments with the appropriate field/season context.	Must
FR6	For cropland ALM fields, record baseline/project practices, production records, livestock schedules, soil sampling plans, strata, samples, laboratory results, custody events, and reviewed SOC evidence required by the supported calculation scope.	Must
FR7	Calculate the implemented VM0051 v1.1 QA3/default-factor rice result deterministically, including supported CH ₄ scaling, uncertainty and irrigation-change N ₂ O terms, while exposing unsupported methodology elements separately from the numerical calculation.	Must
FR8	Calculate the implemented VM0042 v2.2 ALM subset from frozen evidence, including supported SOC, uncertainty, selected non-SOC emission terms, integrated livestock terms, and restricted VMD0054 v1.1 displacement-leakage logic.	Must

ID	Requirement	Priority
FR9	Support experimental Random Forest/XGBoost AWD detectors and managed crop models without treating model predictions or model-versus-threshold agreement as independent scientific validation.	Should
FR10	Provide bundle-scoped generative AI draft explanations through configured providers, using deterministic server-built context packets and server-side validation; AI output must not control accounting values, readiness, or verification decisions.	Should
FR11	Export the formats implemented for each workflow: legacy verification PDF/JSON/CSV, versioned calculation JSON, and snapshot-backed ALM calculation PDF, while clearly distinguishing snapshot-backed outputs from legacy paths whose historical provenance is not fully frozen.	Must
FR12	Provide field/project history and portfolio views that let authorized users inspect registered fields, current methodology path, calculation versions, latest results, and relevant project context.	Should
FR13	Manage versioned methodology documents, bundles, corrections, project applicability, and deterministic readiness requirements for the supported accounting pathways.	Must
FR14	Persist each new calculation as a versioned record with frozen evidence/input/result/readiness context and supersession metadata so later evidence or calculation changes do not silently rewrite historical numerical payloads.	Must
FR15	Allow eligible calculation versions to enter an internal review workflow with submissions, reviewer assignments, review events, findings, comments, and notifications.	Should
FR16	Provide portfolio and operational views for project/calculation summaries, methodology/source coverage, configured AI providers, background jobs, and worker status.	Should
FR17	Enforce organization scoping and project-role permissions for supported workflows while allowing permitted updates to editable project/field metadata without changing immutable accounting-pathway identity.	Must
FR18	Execute long-running signal and AI work through the durable database-backed queue with worker heartbeat, retry, cancellation, and failure states rather than relying on in-request synchronous execution.	Must

2.3 Non-functional Requirements

Table 2: Non-functional Requirements and Verification Targets

ID	Category	Requirement	Verification
NFR1	Security & Identity	Authentication secrets and credentials must be protected; production configuration must reject weak/default JWT secrets, and account tokens must be validated against the current active user and token-version state.	Authentication/account-access tests plus configuration and security-code review.
NFR2	Authorization & Tenant Isolation	Tenant identity must come from authenticated server state, and project/role authorization must be enforced for protected operations; any endpoint that remains organization-scoped rather than uniformly project-member-scoped must be documented as a limitation.	Tenant/auth regression tests and endpoint-level authorization review.
NFR3	Auditability & History	Saved calculations must retain frozen evidence/input/result/readiness context and a version chain sufficient to review historical results without silently recomputing them from current evidence.	Calculation/snapshot/export regression tests and historical-version comparisons.
NFR4	Reproducibility	Deterministic accounting and scoped fingerprints must be reproducible from stored values and stable canonical serialization; software/model versions and	Golden-value tests, fingerprint tests, snapshot/version tests,

ID	Category	Requirement	Verification
		provenance limitations must be disclosed where complete executable provenance is not yet captured.	and code review of version/provenance fields.
NFR5	Methodology Integrity	Unsupported, incomplete, or reviewer-dependent methodology requirements must remain explicit and must not be silently interpreted as satisfied, zero, or externally verified.	Methodology/readiness tests and review of blocking/unsupported requirement behavior.
NFR6	External-Service Resilience	External-service failures (Earth Engine, email, storage, AI provider, worker availability) must degrade through actionable application/job states rather than exposing raw failures as successful results.	Job/API tests, fault-path review, and operational/manual service checks.
NFR7	Availability & Job Durability	Long-running analytics and AI work must be decoupled from request lifetimes through the durable queue, with persisted pending/running/completed/failed/cancelled state, retries where supported, and worker heartbeat visibility.	Durable-job tests, worker CLI checks, and job-state UI/API verification.
NFR8	Maintainability	Domain packages must remain separated by capability (accounts, projects, evidence, methodology, carbon, signals, jobs, AI, persistence, reporting) so API/frontend changes do not become calculation authority.	Architecture/code review plus full regression suite.
NFR9	Portability & Migration Transparency	The system must support local/test SQLite and hosted PostgreSQL configurations without changing domain behavior; deployment-specific limitations and unverified migration paths must be stated explicitly.	Fresh/repeated SQLite initialization tests, representative legacy-upgrade test, and deployment/migration review.
NFR10	Privacy & Storage	Private evidence/model artifacts must be accessed through authenticated application paths, use sanitized storage keys/checksums, and avoid exposing tenant files as public static assets; shared-storage/backup limitations must be disclosed.	Attachment/storage tests, authenticated-download review, and backup/restore script review.
NFR11	Usability & Accessibility	Important UI states—loading, queued, running, blocked, permission denied, validation failure, completed, failed and retry—must be visible and understandable without relying on color alone.	Frontend tests, ESLint, responsive/UI review, and final-state screenshot inspection.
NFR12	Scientific Validity	Satellite/ML outputs must be represented as experimental/supporting evidence unless independently validated; software tests or model-versus-threshold metrics must not be described as proof of real-world AWD accuracy.	Review of detector-label provenance plus explicit scientific-validation status in the report.
NFR13	Deployment & Operations	Supported deployment topology, external dependencies, storage/database configuration, and backup/recovery assumptions must be documented separately from what has actually been verified live.	Deployment checks, configuration review, worker/CLI checks, and explicit verified-vs-supported reporting.
NFR14	Testability	The final verification process must pass the current backend/core regression suite and frontend test/build/type checks, while any unexecuted scientific, browser-E2E, PostgreSQL, provider, or external-service validation remains explicitly separate.	Recorded local verification reports: backend/core 177 passed, 0 failed, 0 skipped; frontend tests passed; production build passed; TypeScript passed; ESLint completed with 0 errors and 5 warnings. These

ID	Category	Requirement	Verification
			commands were not rerun during preparation of this final report.

2.4 Stakeholders and Viewpoints

Table 3: Stakeholders and Viewpoints

Stakeholder	Interest
Carbon Project Developer / Project Lead	Creates and manages project context, registers fields, coordinates evidence, runs scoped calculations, reviews readiness/history, and prepares exports; project contributors/viewers receive narrower role-based access.
Researcher / Data Scientist	Builds/evaluates experimental AWD and crop models, studies satellite/threshold behavior, and distinguishes model agreement from independently validated agronomic accuracy.
Internal Reviewer	Reviews eligible calculation versions inside Terra-Audit, records findings/comments/events, and checks evidence/readiness context; this role is internal and is not a substitute for a VVB.
Program Operator (farmer cooperative / NGO)	Coordinates enrollment and field/evidence collection across participating farms and uses the platform to prepare project records for downstream review.
Farmer / Landholder	Owns or manages enrolled land and provides or confirms agronomic, production, livestock, soil, irrigation, and other field evidence used by project staff.
System Operator / Administrator	Maintains application configuration, methodology sources, provider permissions, worker/queue operations, database/storage settings, and operational/backup tooling.
External Validation/Verification Body (VVB) / Verra Process	Independently evaluates methodology applicability, evidence, monitoring, calculations, safeguards and verification requirements outside the application. Terra-Audit internal review and estimated quantities do not constitute VVB verification or Verra-issued VCUs.
External Data and AI Services	Google Earth Engine/Copernicus provide satellite observations, while configured AI/email/storage services support optional platform functions; their availability and credentials constrain runtime behavior.

2.5 Quality Function Deployment and Prioritization

The final QFD groups the functional requirements in Table 1 by user value without duplicating their full specification. Normal requirements are the essential access, evidence, accounting, readiness, versioning, export and durable-job capabilities (FR1, FR2, FR5-FR8, FR11, FR13, FR14, FR17, FR18). Expected requirements cover traceable signal processing, history/portfolio, internal review and operational visibility (FR3, FR4, FR12, FR15, FR16). Exciting requirements are the experimental ML and bounded generative-AI capabilities (FR9, FR10), which remain outside deterministic accounting and external verification authority.

Traceability is stakeholder need → QFD category → FR ID/priority → implementing component in Chapter 5 → verification evidence in Chapter 7. This preserves examiner-visible prioritization while avoiding a second copy of the requirements specification.

2.6 Representative User Stories

US1 — Project lead: create/select project context, register fields, assign members, organize evidence and save versioned calculations so every result remains tied to an authorized project and field.

US2 — Rice/ALM project user: collect the pathway-specific signal or structured practice/SOC/production/leakage evidence needed for the scoped VM0051 or VM0042 calculation, while retaining source context and blocking incomplete requirements.

US3 — Reviewer: inspect methodology readiness, frozen calculation versions, evidence context, findings and history so unresolved requirements remain visible and later evidence does not silently rewrite historical numerical results.

US4 — Researcher or authorized AI user: compare experimental threshold/ML outputs or request grounded draft explanations while keeping model agreement and AI text distinct from scientific validation, accounting authority and external verification.

US5 — Administrator: monitor methodology sources, provider permissions, jobs and worker state so background/external-service failures can be diagnosed without being presented as successful work.

3. System Modeling

3.1 Use Case Diagram and Descriptions

Use-case modeling describes the externally visible goals that current Terra-Audit actors achieve through the system. The final model reflects the implemented multi-user web application rather than the retired Streamlit prototype. Primary actors are project users, researchers, internal reviewers, and administrators. Supporting systems include Google Earth Engine, the application database/private storage, and configured AI/email services. External VVB/Verra processes remain outside Terra-Audit's authority boundary.

Table 4: Use-Case Inventory and Actor Mapping

Scope / ID	Use Case	Primary Actor	Secondary Actor	Figure
Context	Terra-Audit	Project User, Researcher, Internal Reviewer, Administrator	Earth Engine; AI Provider	Fig 1
Detail	Detailed Use-Case Model	Project User, Researcher, Internal Reviewer, Administrator	—	Fig 2
1.1	Project & Field Management	Project Lead / Contributor	Accounts; Projects; Persistence	—
1.2	Evidence & Season Management	Project User / Internal Reviewer	Evidence; Attachments; Soil/Production	—
1.3	Signal Analytics	Project User / Researcher	Earth Engine; Jobs/Worker	—
1.4	Carbon Calculation	Project User	VM0051/VM0042 engines; Snapshot; Leakage	Fig 3
1.5	Methodology Readiness	Project User / Internal Reviewer	Methodology Registry; Readiness	—
1.6	Internal Review	Project Lead / Internal Reviewer	Review Submissions; Findings	—
1.7	Reporting & Export	Project User / Internal Reviewer	Reporting; Calculation History	—
1.8	AI Assistance & Research	Researcher / Authorized User	AI Providers; ML/Managed Models	—
1.9	Administration & Operations	Administrator	Methodology Sources; Jobs; Workers; Providers	—

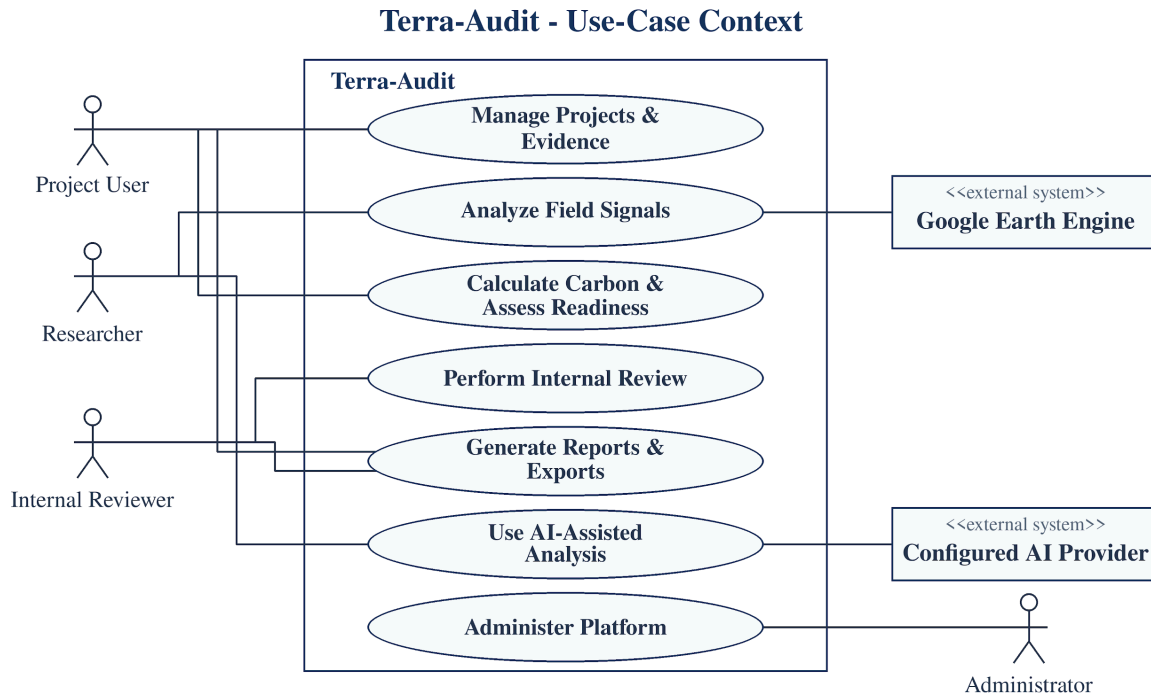


Fig 1: Terra-Audit use-case context. Human actors and external Earth Engine/AI services connect to high-level use cases inside the system boundary; internal database/storage is not modeled as an actor.

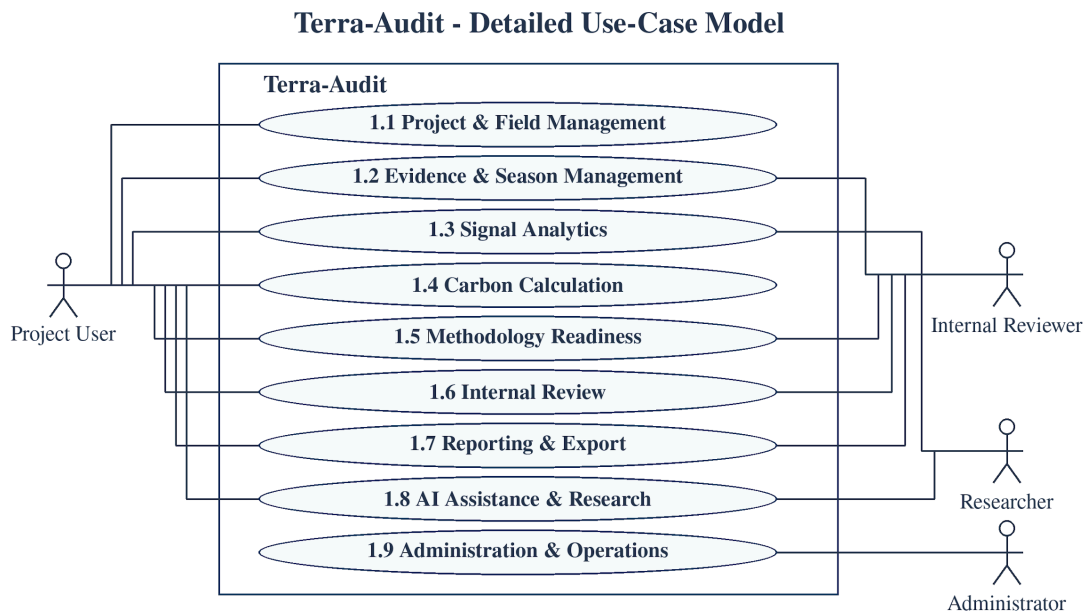


Fig 2: Terra-Audit detailed use-case model. The nine user-goal areas are project/field management, evidence/season management, signal analytics, carbon calculation, methodology readiness, internal review, reporting/export, AI assistance/research, and administration/operations. Actor participation is shown explicitly; Table 4 provides the corresponding role details.

Use Case 1.4 - Carbon Calculation

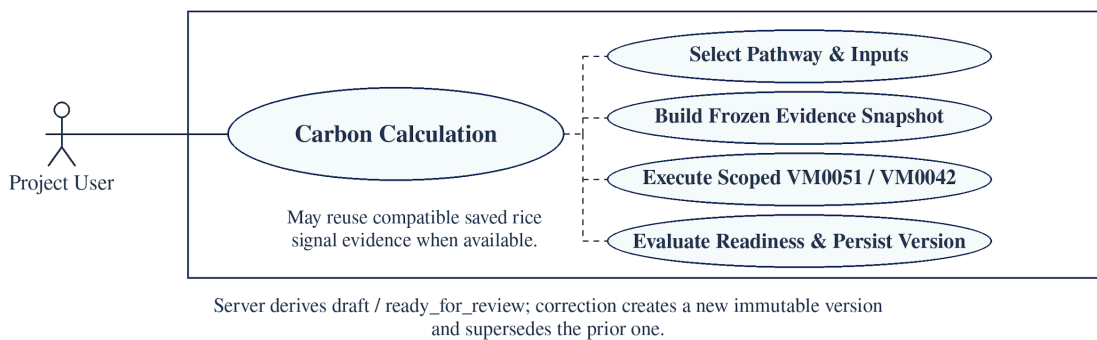


Fig 3: Carbon Calculation (Use Case 1.4). The user selects the accounting pathway and inputs, optionally reuses compatible saved signal evidence for rice, builds the evidence snapshot, executes the scoped VM0051 or VM0042/leakage logic, evaluates gates/readiness, and saves a calculation version.

The nine use cases separate user goals from implementation details. For example, durable jobs, evidence snapshots, methodology bundles, and provider adapters are internal collaborators rather than user goals by themselves. The following specifications summarize actor, trigger, normal flow, alternative/error behavior, and postcondition for each detailed use case.

3.1.1 Use Case Descriptions

The retained context, detailed use-case and Carbon Calculation diagrams give the visual overview; Table 5 preserves the actor, precondition/trigger, normal flow, alternative/error flow and postcondition for all nine detailed use cases.

Table 5: Detailed Use-Case Specifications

Use Case	Actor(s)	Precondition / Trigger	Normal Flow	Alt. / Error Flow	Postcondition
Project & Field Management	Project Lead / Contributor	Authenticated organization/project context.	Create/select project → manage members → register field/geometry → assign project/farm membership → update permitted metadata.	Invalid geometry, duplicate/invalid input, or insufficient permission → no write and actionable error.	Authorized project/field context persisted.
Evidence & Season Management	Project User / Internal Reviewer	Field exists; authorized evidence context.	Create/correct season → observations/practice events → attachments → soil/lab/custody → production/livestock evidence.	Validation, permission, storage, or missing-required-data errors remain visible; corrections append history rather than silently replacing prior versions.	Versioned evidence state available to readiness/calculation workflows.
Signal Analytics	Project User / Researcher	rice_awd field and monitoring window selected.	Reuse compatible active/cache result where possible → otherwise enqueue signal_run → worker queries Earth Engine → process	Earth Engine, worker, empty-data, detector, or authorization failure → job/error state;	Completed signal result or explicit failed/cancelled state is available.

Use Case	Actor(s)	Precondition / Trigger	Normal Flow	Alt. / Error Flow	Postcondition
			threshold/ML detector → store result.	no successful result is fabricated.	
Carbon Calculation	Project User	Registered field plus pathway-specific inputs/evidence.	Select pathway and inputs → optionally use saved signal → build snapshot → run VM0051 or VM0042/leakage → evaluate gates/readiness → save version.	Invalid inputs, unsupported/applicability conditions, leakage/SOC hard blocks, or unresolved readiness prevent a ready-for-review state.	Versioned draft or ready-for-review calculation is persisted.
Methodology Readiness	Project User / Internal Reviewer	Active methodology bundle and project/field context.	Resolve bundle/applicability → evaluate registered requirements/corrections → calculate fingerprint/readiness → expose blockers and reviewable items.	Unsupported or automated-only requirements cannot be silently overridden; stale evidence changes the fingerprint/state.	Current readiness and blocking evidence context is displayed/stored.
Internal Review	Project Lead / Internal Reviewer	Specific calculation version is ready_for_review.	Submit calculation → assign reviewer → record review events/findings/comments → resolve or track findings.	Blocked/draft calculation cannot enter normal review; authorization failures are rejected.	Version-pinned internal review trail is retained.
Reporting & Export	Project User / Internal Reviewer	Calculation/history context exists and user is authorized.	Select the relevant verification or calculation record → choose an export format actually available for that record → inspect history/portfolio context → download.	Unsupported export, missing context, or legacy provenance limitation is disclosed rather than silently substituted.	Supported evidence/report artifact is downloaded with stated provenance limits.
AI Assistance & Research	Researcher / Authorized User	Authorized project context plus evidence/methodology context or a research dataset.	Build/evaluate AWD or crop models, or request AI explanation → build deterministic packet → provider/job → validate citations/numbers → retain draft.	Missing provider/permission/data, oversized packet, or failed validation → warning/failed job; no authoritative decision is written.	Research metrics or a validated human-review draft is available.
Administration & Operations	Administrator	Administrative authorization.	Manage methodology sources/provider permissions → inspect jobs/workers → retry/cancel supported jobs → use operational/backup tooling.	Permission, stale worker, external-service, or configuration failure remains visible and does not masquerade as success.	Operational configuration and monitoring state are updated or an explicit failure remains.

3.2 Activity Diagram

The activity models below describe the final implemented control flow rather than the midterm Streamlit sequence. They emphasize authentication/project context, structured evidence, durable signal/AI jobs, deterministic pathway calculations, methodology readiness, versioned snapshots, internal review, and export. Error and blocked states are represented explicitly so that failed external services or incomplete methodology requirements do not appear as successful MRV outcomes.

Terra-Audit - Overall Activity Flow

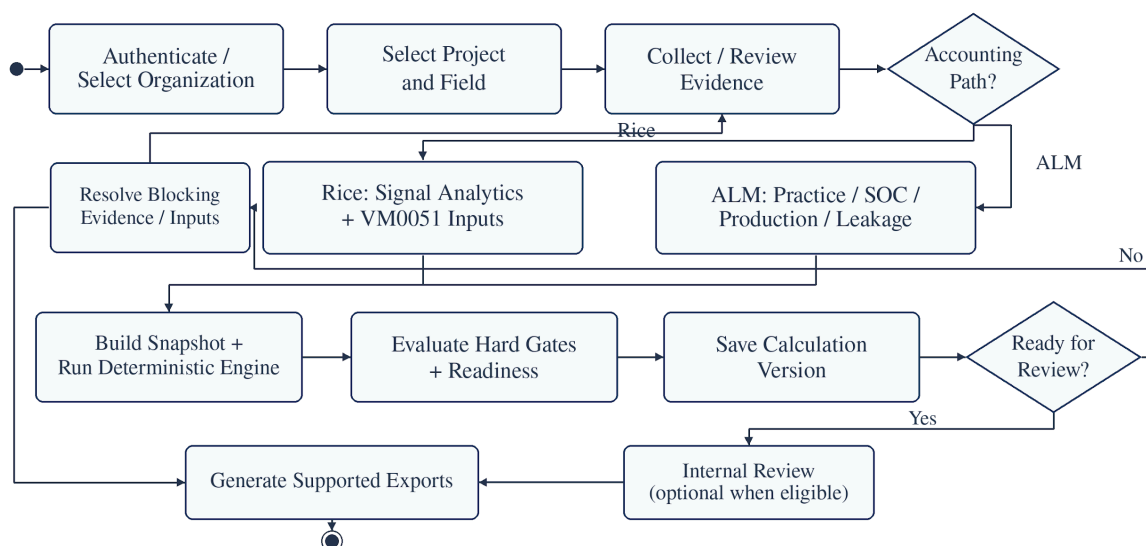


Fig 4: Terra-Audit overall activity flow. After authentication and project/field selection, evidence is collected, the workflow branches to the rice or ALM accounting path, readiness/hard gates are evaluated, a versioned calculation snapshot is saved, eligible versions may enter internal review, and supported reports/exports are generated.

Versioned Carbon Calculation - Activity

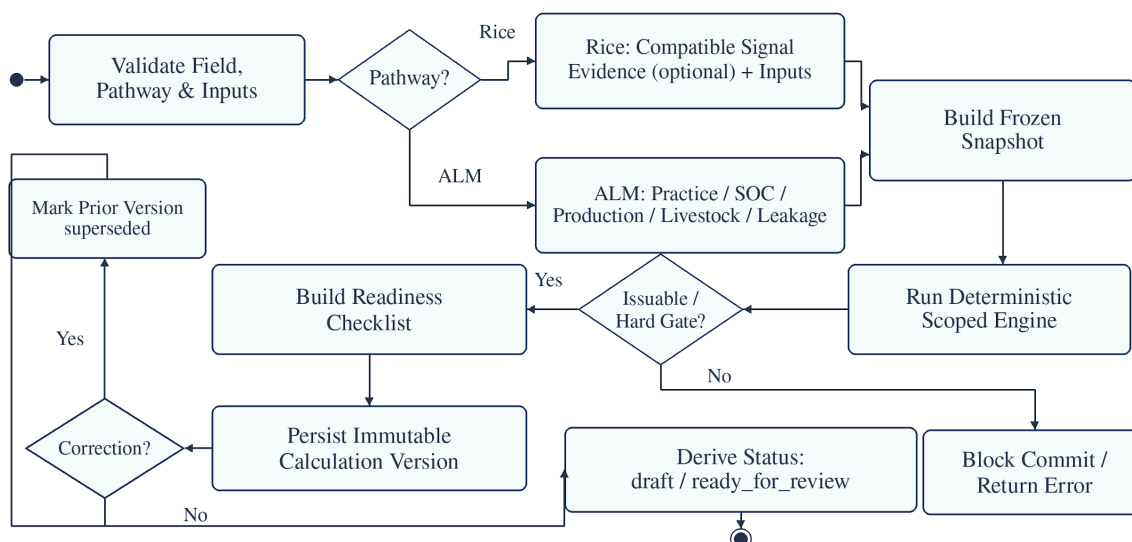


Fig 5: Versioned Carbon Calculation activity. Rice calculations may use compatible saved signal evidence or manual inputs; ALM calculations resolve practices, SOC, production, livestock, and leakage evidence. Both paths build a snapshot, run deterministic scoped engines, evaluate hard gates/readiness, and save a new calculation version rather than rewriting historical results.

Methodology Readiness & Internal Review - Activity

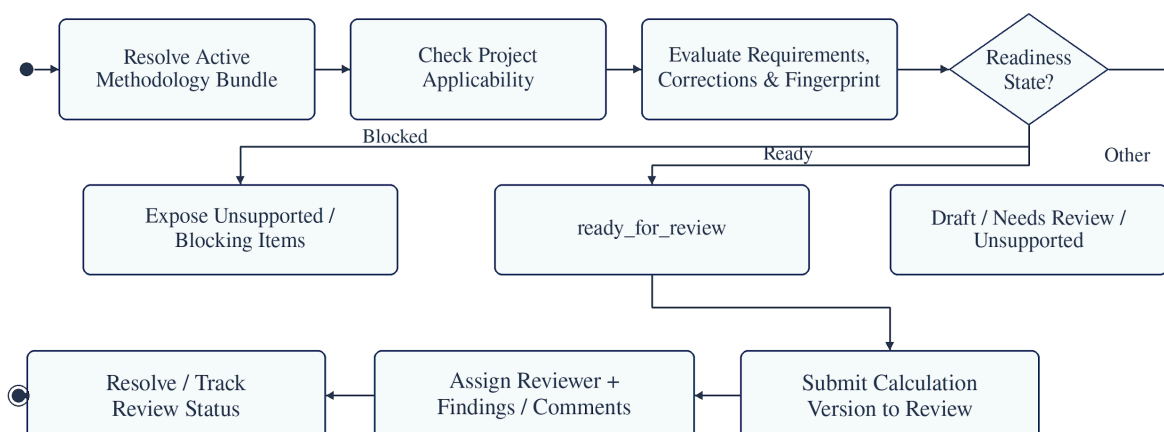


Fig 6: Methodology Readiness and Internal Review activity. Terra-Audit resolves the active bundle, applicability, evidence fingerprint, corrections, and requirement statuses. Only ready_for_review calculation versions can enter the internal review workflow; blocked or unsupported states remain explicit.

3.2.1 Cropland ALM Detailed Activity Flow

The final ALM flow begins with an organization/project-scoped cropland_alm_vm0042 field and dated project membership. Users record baseline/project crop seasons and practice schedules, optional supported livestock schedules, production records, soil sampling plans/strata/samples, laboratory results, custody events, and reviewed SOC mappings (or explicitly identified legacy aggregate SOC evidence). A project/bundle/reporting-period leakage assessment is then saved. The calculation workflow builds a frozen snapshot containing the relevant practice, production, leakage and SOC evidence; the restricted VMD0054 v1.1 adapter computes the supported displacement-leakage term; and the scoped VM0042 v2.2 engine evaluates SOC/non-SOC terms, uncertainty, allocation and buffer logic. Readiness and engine hard gates are evaluated before a versioned calculation is persisted. Only a ready_for_review version can enter internal review. Snapshot-based ALM PDF and calculation JSON exports can then be generated. Current restrictions—including single-field leakage scope, unsupported mitigation Step 2, nonannual SOC blocking where unresolved annualization applies, and incomplete sequential-period cumulative-state reconciliation—must remain visible rather than being represented as complete methodology coverage.

4. Data & Information Modeling

4.1 Entity-Relationship Model

Terra-Audit uses a broad, domain-oriented persistence model rather than the smaller midterm schema. To keep the final report readable, Section 4.1 retains one high-level ERD plus a compact relationship summary, while Appendix A contains the consolidated technical ERD. Relationships are logical/application relationships unless explicitly stated; the current SQLite schema does not generally declare FOREIGN KEY constraints, so scoped identifiers, application queries, authorization checks, and selected indexes or constraints carry the corresponding integrity responsibility. Table 6 remains the concise textual schema inventory.

4.1.1 High-Level Entity Relationship Diagram

Fig 7 provides an at-a-glance view of the persistent model verified against the final implementation. It groups runtime persistence entities by logical domain and shows entity names and cardinalities without full column detail. Section 4.1.2 summarizes the principal business relationships, while Appendix A provides the consolidated technical view.

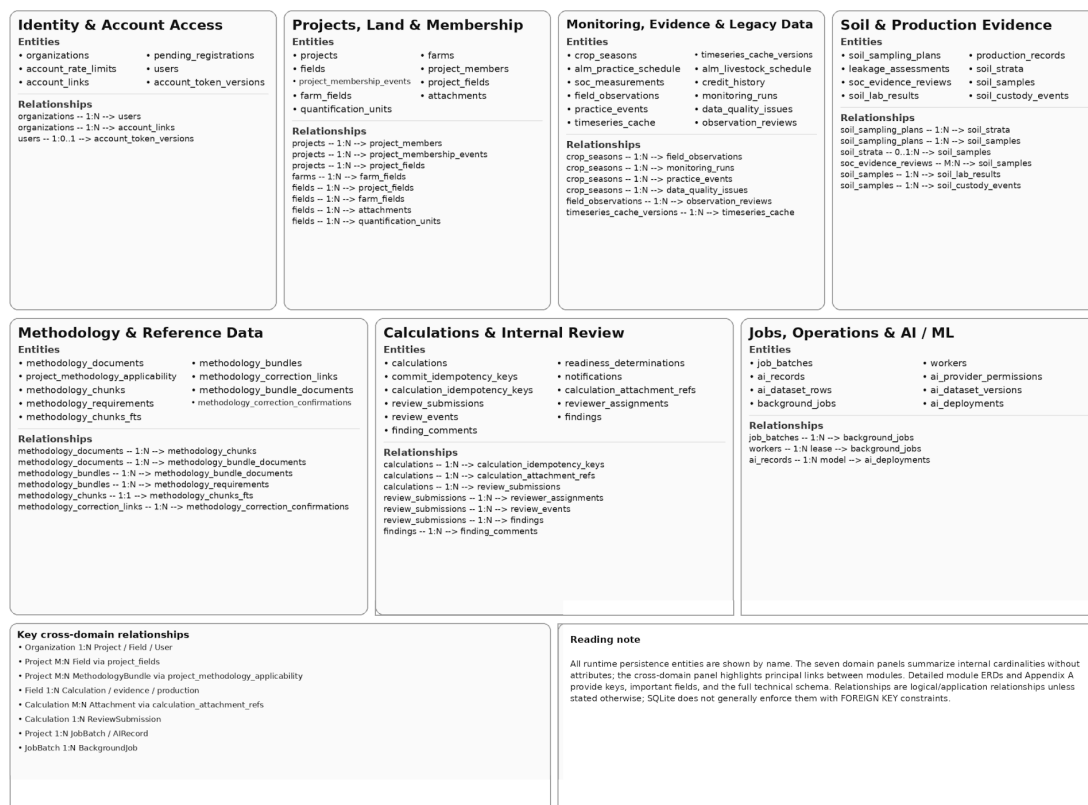


Fig 7: High-level Terra-Audit entity-relationship model. All runtime persistence entities are grouped by domain; the overview shows entity names and cardinalities without column detail.

4.1.2 Focused Relationship Summary

To avoid repeating the same schema across five large diagrams, this section preserves the principal relationships in compact form; Appendix A retains the consolidated technical ERD.

Identity and project context — User-to-project access is represented through ProjectMember with explicit roles. Project-to-field membership is represented through effective-dated ProjectField records, so a field is not reduced to a single project_id and assignment history can be retained.

Evidence and sampling — CropSeason, FieldObservation, ObservationReview and PracticeEvent use append-oriented field/season records. Attachments remain field-scoped and reference supported targets. The ALM soil path links sampling plans, optional strata, samples, laboratory results, custody events and reviewed SOC mappings; production records and leakage assessments provide additional field/project evidence.

Calculation provenance — Calculation records are versioned and immutable at the application level: chain_id and supersedes_calculation_id preserve correction history, while snapshot_json,

inputs_json, result_json and readiness_json freeze the committed context.

CalculationAttachmentRef retains protected attachment links; legacy credit history remains distinguishable from snapshot-backed calculations.

Methodology reference data — MethodologyDocument, MethodologyBundle, bundle-document membership and MethodologyRequirement form the shared registry. Tenant-specific ProjectMethodologyApplicability records bind a project/pathway to a bundle and effective period; correction/confirmation records remain separate from global source data.

Internal review — ReviewSubmission pins one immutable calculation version. Reviewer assignments, lifecycle ReviewEvents, Findings and finding comments/resolution activity belong to the internal workflow and never represent external VVB verification, Verra registration or registry issuance.

4.2 Database Schema

Table 6: Selected Persistent Database Schema

Domain	Important Entities / Tables	Purpose
Accounts & Identity	organizations, users, pending_registrations, account_links, account_token_versions, account_rate_limits	Tenant identity, account access/recovery, token revocation/versioning, and account-level throttling.
Projects & Land	projects, project_members, project_membership_events, farms, fields, project_fields, farm_fields, quantification_units	Project roles and dated land/field memberships; field records carry the registered accounting pathway and geometry context.
Monitoring & Evidence	crop_seasons, field_observations, observation_reviews, practice_events, monitoring_runs, data_quality_issues, attachments	Versioned crop-season history and structured monitoring/evidence records, including private evidence-file metadata.
Soil / Production / ALM	soil_sampling_plans, soil_strata, soil_samples, soil_lab_results, soil_custody_events, soc_evidence_reviews, alm_practice_schedule, alm_livestock_schedule, production_records, leakage_assessments	Structured ALM evidence for reviewed SOC, practices, production, supported livestock inputs, and restricted leakage assessment.
Calculations & Review	calculations, calculation_idempotency_keys, calculation_attachment_refs, readiness_determinations, review_submissions, reviewer_assignments, review_events, findings, finding_comments	Versioned calculation snapshots/results/readiness plus version-pinned internal review and findings history.
Methodology	methodology_documents, methodology_bundles, bundle-document mappings, methodology_requirements, project_methodology_applicability, methodology_chunks/FTS, correction links/confirmations	Methodology/version/correction registry, retrieval index, applicability context, and requirement definitions used by readiness and AI context.
Jobs / AI / Satellite	background_jobs, job_batches, workers, ai_records, ai_deployments, ai_provider_permissions, timeseries_cache,	Durable asynchronous work, worker state, satellite cache/versioning, provider/model records, and lazily created research-dataset tables.

Domain	Important Entities / Tables	Purpose
	timeseries_cache_versions; lazy ai_dataset_rows / ai_dataset_versions	

The SQLAlchemy connection layer supports SQLite and PostgreSQL. SQLite startup initialization contains historical additive ALTER TABLE operations and selected rebuild/copy migrations; the PostgreSQL path uses final-shape DDL plus module-specific additive changes. There is no Alembic-style versioned migration ledger. In the recorded final software checks, fresh SQLite initialization, repeated initialization/restart, a representative legacy SQLite upgrade, and SQLite integrity checking passed. Remote PostgreSQL migration/runtime verification was not completed, and the existing SQLite-to-PostgreSQL transfer script covers only a legacy subset of the current schema.

4.3 Dataset Description (AI-Assisted Detection)

The classical AWD dataset is built from cached rice time-series and labels generated by the rule-based threshold detector rather than by independent field observations. Feature engineering uses satellite/time-series values while excluding the gate's own decision flags as direct predictor features. Random Forest and XGBoost therefore measure model-versus-threshold agreement; their metrics cannot be interpreted as accuracy against independently observed irrigation events.

The implemented evaluation harness now requires field identifiers and uses leakage-safe GroupKFold splits so the same field is not placed in both training and validation folds. Pooled out-of-fold precision, recall, F1, confusion-matrix and ROC-style outputs are still explicitly labeled as threshold-gate agreement. Persisted model bundles also record the satellite processing version and field-grouped split strategy so stale feature versions are rejected during loading.

The principal validation risk is no longer observation-level fold leakage; the current implementation uses field-grouped evaluation. The larger limitation is label provenance: the target labels still come from the same threshold detector that the models are intended to imitate. Sparse drydown events, Sentinel revisit gaps, phenology effects, and the absence of independently observed water-level/irrigation events limit how strongly the resulting metrics can be interpreted.

Future scientific validation should compare detected flood/drydown events against independent ground truth such as water-level-tube observations, irrigation logs, or partner-verified field observations and report event-level precision/recall, timing error, false drydown rate, and performance across seasons/locations. Managed crop models are a separate exploratory feature using reviewed crop-identity labels; neither that work nor the AWD classifiers establish methodology-authoritative AWD compliance.

5. Software Design

5.1 Component Architecture

The final architecture is a multi-user web system rather than the original Streamlit monolith. A Next.js frontend communicates with FastAPI through an authenticated server-side proxy. FastAPI routers and schemas orchestrate shared Python domain packages for accounts, projects, evidence, methodology, carbon accounting, signals, jobs, AI, persistence, and reporting.

Long-running signal and AI operations are persisted in the database-backed job queue and executed by a separate Python worker. Deterministic carbon accounting, readiness and server validation remain authoritative; ML detectors and generative AI are assistive or experimental components.

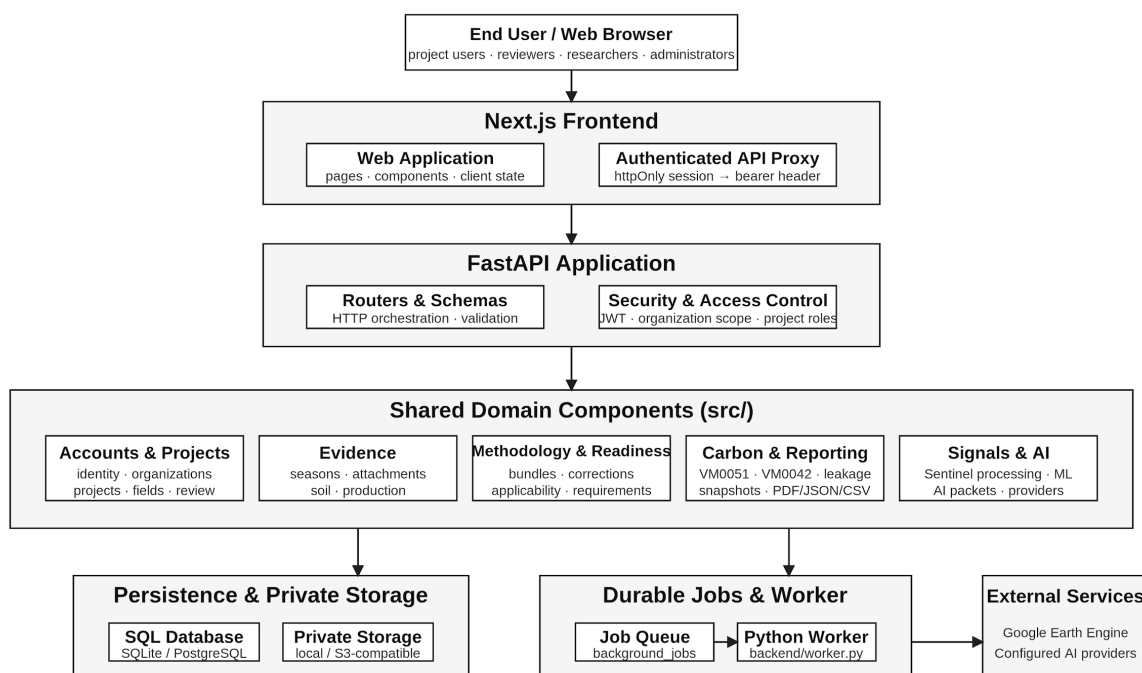


Fig 8: Terra-Audit component architecture. The browser/Next.js/FastAPI request path is separated from the shared domain layer, persistence/storage, durable job queue, worker, external Earth Engine/provider calls, and reporting. The deterministic domain/API layer remains the authority for accounting and readiness; ML and generative AI do not replace that boundary.

Table 7: Component Responsibilities

Component	Responsibility
Next.js Frontend & API Proxy	Renders authenticated pages/components, manages client state/queries, and forwards browser requests through the server-side authenticated API proxy rather than exposing bearer tokens directly to client code.
FastAPI Routers & Schemas	Own HTTP orchestration, request/response validation, route composition, and handoff to shared domain packages.
Accounts / Security / Authorization	Implements identity, password/account-recovery flows, JWT validation, token-version checks, organization scoping, and project/admin access gates.
Projects & Evidence	Manages projects, farms, fields, memberships, crop seasons/corrections, observations, attachments, soil/SOC, production, and internal-review domain records.
Methodology & Readiness	Maintains methodology documents/bundles/corrections, project applicability, requirement definitions, retrieval chunks, evidence fingerprints, and deterministic readiness states.
Carbon Accounting & Leakage	Runs scoped VM0051 v1.1 rice and VM0042 v2.2 ALM calculations plus the restricted VMD0054 v1.1 leakage adapter; AI does not own accounting values.
Snapshots & Calculation Versioning	Builds frozen calculation context, stores inputs/results/readiness, manages idempotency and supersession/version chains, and preserves historical numerical payloads under normal application updates.

Component	Responsibility
Signals / Earth Engine	Queries and processes Sentinel-1 observations, maintains compatible cache versions, applies threshold/phenology logic, and supports selected experimental ML detectors.
Durable Jobs & Worker	Persists long-running work in the database-backed queue; separate worker processes execute signal, AI, methodology-ingestion and related jobs with heartbeat/retry/cancellation state.
AI / ML / Providers / Validation	Supports classical AWD ML, managed crop models, deterministic AI context packets, provider adapters, structured-response validation, and human-review draft explanations.
Reporting & Export	Generates legacy verification PDF/JSON/CSV artifacts, versioned calculation JSON, snapshot-backed ALM calculation PDF, and history/portfolio outputs, with different provenance guarantees for snapshot-backed and legacy paths.
Persistence & Storage	Provides SQLAlchemy-backed SQLite/PostgreSQL access plus local/private S3-compatible artifact storage; application logic enforces relationships because the verified SQLite schema declares no foreign keys.

5.2 Design Classes and Responsibility Boundaries

The implementation is modular but is not a uniform class/service/repository hierarchy. Design responsibilities are distributed across domain packages and a smaller number of classes/helpers. Important software-design units include account/authentication primitives in `src/accounts` and `backend/security.py`; project and internal-review repositories in `src/projects`; evidence modules for seasons, monitoring, soil and production; methodology registry/library/readiness modules; deterministic rice and ALM carbon engines; the leakage adapter; snapshot/calculation-version persistence; signal acquisition/processing; the durable job queue and worker handlers; AI packets/providers/validators and ML models; reporting; and persistence/storage.

The main authority boundary is deliberate: UI, jobs, satellite models and language models may provide inputs, analytics or explanations, but accounting values, readiness states and validation rules remain server-owned deterministic behavior. This separation reduces the risk that a probabilistic component silently changes methodology logic.

5.2.1 CRC Model

CRC cards summarize what each design class is responsible for and which collaborators it depends on. Only implementation-backed classes are shown as CRC cards; supporting module-level services are listed as collaborators rather than being presented as artificial classes. Method-level operations remain in Section 5.2.2 to avoid duplicating implementation detail inside the CRC model.

SpatialDataEngine		AdaptiveAWDGate	
Responsibilities	• Retrieve Sentinel-1 GRD observations • Derive VV/VH, cross-ratio and RVI series • Return cleaned monitoring time series	Responsibilities	• Detect flooded states and drydown events • Add Z-score and signal-difference fields • Extract sowing and harvest indicators
Collaborators	• Google Earth Engine • Signal processing / cache • AdaptiveAWDGate	Collaborators	• SpatialDataEngine • Signal job handlers • Experimental AWD ML

CarbonAssetEngine		AlmCarbonEngine	
Responsibilities	• Execute scoped VM0051 QA3 calculation • Compute CH₄/N₂O and uncertainty intermediates • Block unsupported QA3-size pathway	Responsibilities	• Execute scoped VM0042 ALM calculation • Compute SOC and non-SOC ER/CR terms • Apply leakage allocation, buffer and blocking flags
Collaborators	• AdaptiveAWDGate • Snapshot / versioning modules • Methodology readiness	Collaborators	• AlmPracticeValidator • VMD0054 leakage adapter • Snapshot / versioning modules • Methodology readiness

Fig 8A: CRC cards for core Terra-Audit design classes. Each card lists implementation-backed responsibilities and collaborators; method-level operations are detailed separately in Section 5.2.2.

5.2.2 Implementation Class and Method Mapping

SE801 requires explicit identification of design classes. Terra-Audit is capability-oriented rather than a uniform object hierarchy, so the mapping below names the real classes and module-level operations that own the corresponding behavior in the final implementation.

1. CarbonAssetEngine — `src/carbon/rice.py`, `calculate_credits()`: AWD events, season length, area, nitrogen, pre-season regime and amendment inputs → deterministic VM0051 intermediate values, hard-gate state and scoped rice result.
2. AlmCarbonEngine — `src/carbon/alm.py`, `calculate_credits()`: practices, SOC measurements, area, verification period, livestock, cumulative state and frozen leakage evidence → VM0042 intermediate values, uncertainty/allocation/buffer terms, blocking flags and scoped ALM result.
3. SpatialDataEngine — `src/signals/earth_engine.py`, `extract_clean_timeseries()`: field geometry and monitoring dates → supported Sentinel-1 time series used by downstream rice analytics.
4. AdaptiveAWDGate — `src/signals/threshold_gate.py`, `analyze_irrigation_behavior()` and `extract_phenology()`: processed signal series → flooded/drydown and sowing/harvest indicators used as the deterministic reference path for experimental AWD models.
5. Snapshot and calculation versioning — `src/carbon/snapshot.py` `build_snapshot()/calculate_from_snapshot()` and `src/carbon/calculations.py` `commit_calculation()`: current evidence and inputs → frozen calculation context, deterministic result, readiness, idempotency state and immutable version/supersession metadata.
6. Methodology readiness — `src/methodology/readiness.py`, `build_readiness_checklist()`: field/pathway/evidence/bundle context → satisfied, missing, needs-review, not-applicable or unsupported requirement states plus bundle context.
7. Internal review — `src/projects/reviews.py`, `transition()`: version-pinned submission, requested state, actor and optimistic version → server-validated internal-review transition and persisted events/findings state.
8. Durable jobs — `src/jobs/queue.py` `create_job()`, `claim_next_job()`, `complete_job()`, `fail_job()` and cancellation/heartbeat helpers: long-running work request → persisted pending/running/done/error/cancel-requested/cancelled lifecycle.

9. AttachmentStorage — `src/persistence/storage.py` `save()`, `open()`, `delete()`: private artifact key/file → storage reference; `LocalFilesystemStorage` and `S3Storage` provide the concrete local and private S3-compatible implementations.

10. AI packet/validation boundary — `src/ai/packets.py` `build_packet()` and `src/ai/validate.py` `validate_response()`: authorized action and deterministic project/methodology context → bounded provider packet and validated human-review draft; AI cannot write accounting or readiness decisions.

11. Reporting — `src/reporting/reports.py` generation functions: stored verification/calculation/evidence context → the export formats implemented for the applicable legacy or snapshot-backed workflow.

The browser request boundary is separate: `frontend/lib/api.ts` sends actions through the `Next.js` catch-all API proxy, which reads the `httpOnly` session server-side and forwards an authenticated request to FastAPI. This keeps browser UI state outside the accounting and authorization authority boundary.

5.3 Persistent Data Sources and Interfaces

The primary transactional store is accessed through SQLAlchemy and supports SQLite for local/test use and PostgreSQL for hosted use. Private evidence attachments and managed-model artifacts can use local filesystem storage or a private S3-compatible backend. Satellite observations are retained in versioned time-series cache tables; methodology source documents are indexed into document/chunk records; background jobs and worker heartbeat state are persisted in the same database-backed queue. These sources are accessed through application modules rather than by the frontend directly.

5.3.1 Context-Level Data Flow (DFD Level 0)

At Level 0, Terra-Audit is treated as one process. The diagram shows the external actors/services and major information exchanges only; internal data stores and module-level processes are deferred to Level 1.

Terra-Audit - Context DFD (Level 0)

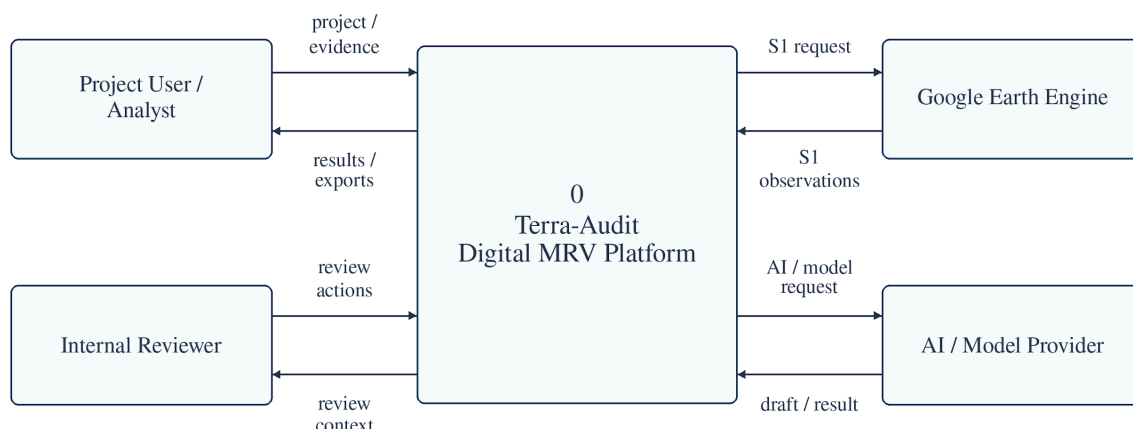


Fig 8B: Terra-Audit Level-0 data-flow diagram showing the major external actors/services and information exchanged with the platform.

5.3.2 Level-1 Data Flow (DFD Level 1)

Level 1 decomposes the context process into five implementation-backed processes and four logical data stores while preserving the external flows shown in Fig 8B.

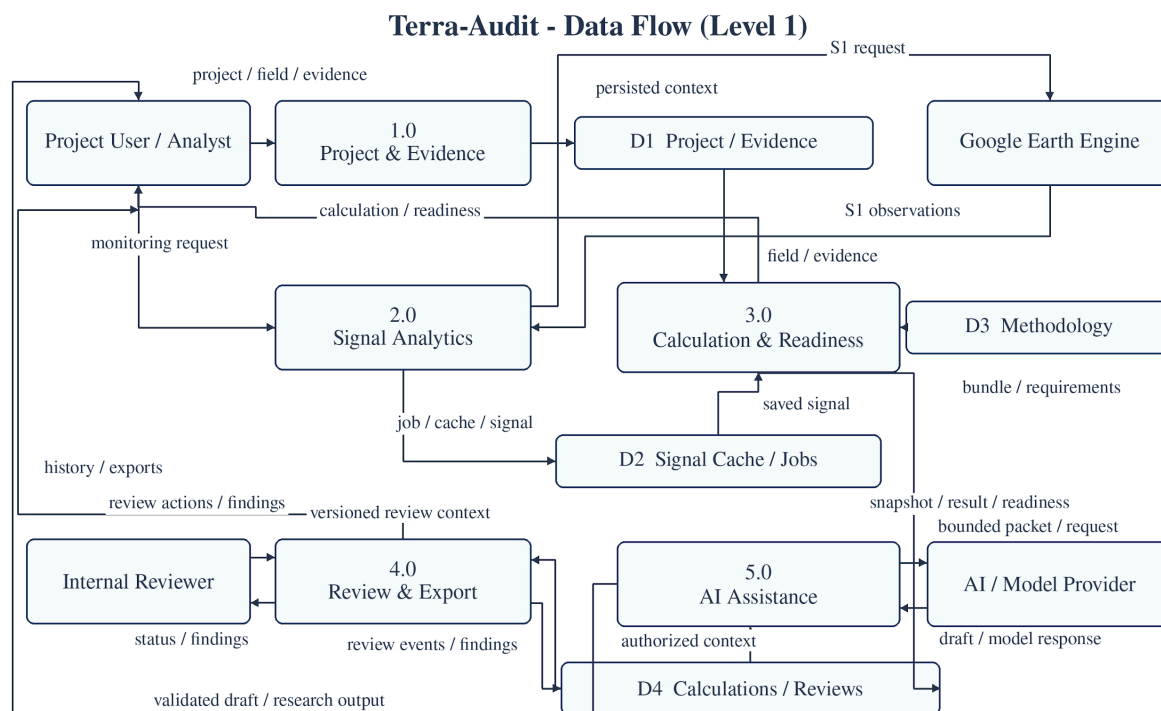


Fig 8C: Terra-Audit Level-1 data-flow diagram showing implementation-backed processes, logical data stores and external-service flows.

5.4 Behavioral Design

The behavioral design combines the event table with persisted state machines in Figs 9(a)-9(b) and implementation-backed sequence diagrams in Figs 9(c)-9(d). Together they trace triggers, server-owned state changes, and the principal request-to-persistence interactions.

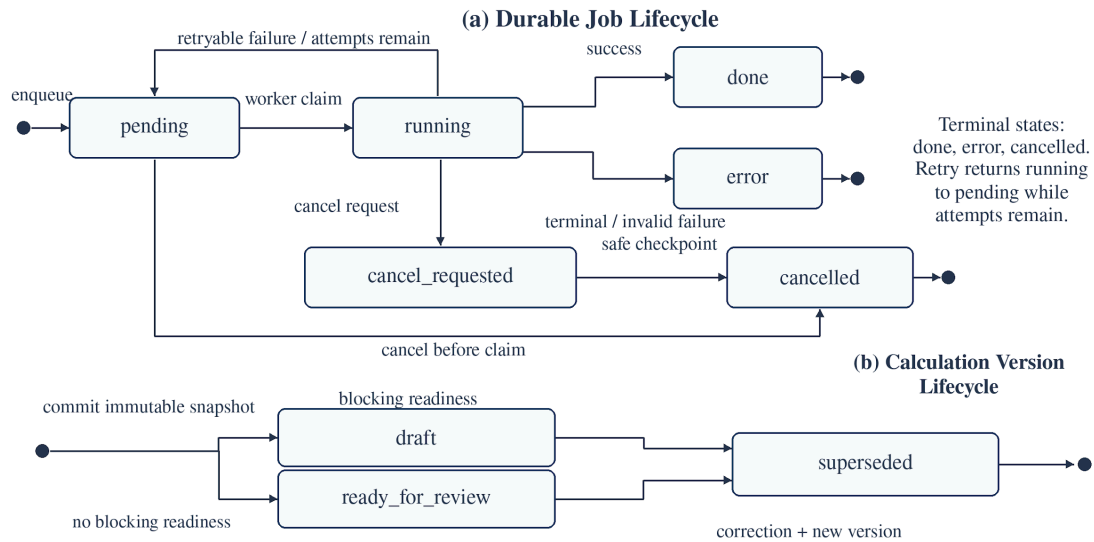
5.4.1 Behavioral Event Table

The implementation-backed events below cover durable signal work, immutable calculation commits, version-pinned internal review and data-quality issue handling.

Event	Trigger / precondition	System response	Result / state
Signal run	User/researcher; rice field + monitoring window	Reuse compatible work or create signal run; worker gets Sentinel-1 and stores processed result	pending → running → done / error / cancelled
Calculation commit	User; valid inputs + stable evidence snapshot	Freeze snapshot; execute engine + readiness; persist new version	draft or ready_for_review; correction → superseded
Review submission	Calculation = ready_for_review	Create version-pinned submission + initial review event	submitted
Review action / finding	Authorized reviewer; current submission/version	Record finding/comment or allowed transition; blocking findings prevent approval	in_review / changes_requested / internally_approved / rejected / withdrawn
Data-quality issue	Issue detected/open	Upsert issue; acknowledge or resolve with actor/reason	open → acknowledged → resolved

Table 7A: Implementation-backed behavioral event table for Terra-Audit.

Persisted States - Jobs and Calculation Versions



Status is derived server-side on commit. A correction creates a new immutable calculation version; the prior version becomes superseded.

Fig 9(a): Persisted state transitions for durable jobs and immutable calculation versions.

Persisted States - Internal Review and Data-Quality Issues

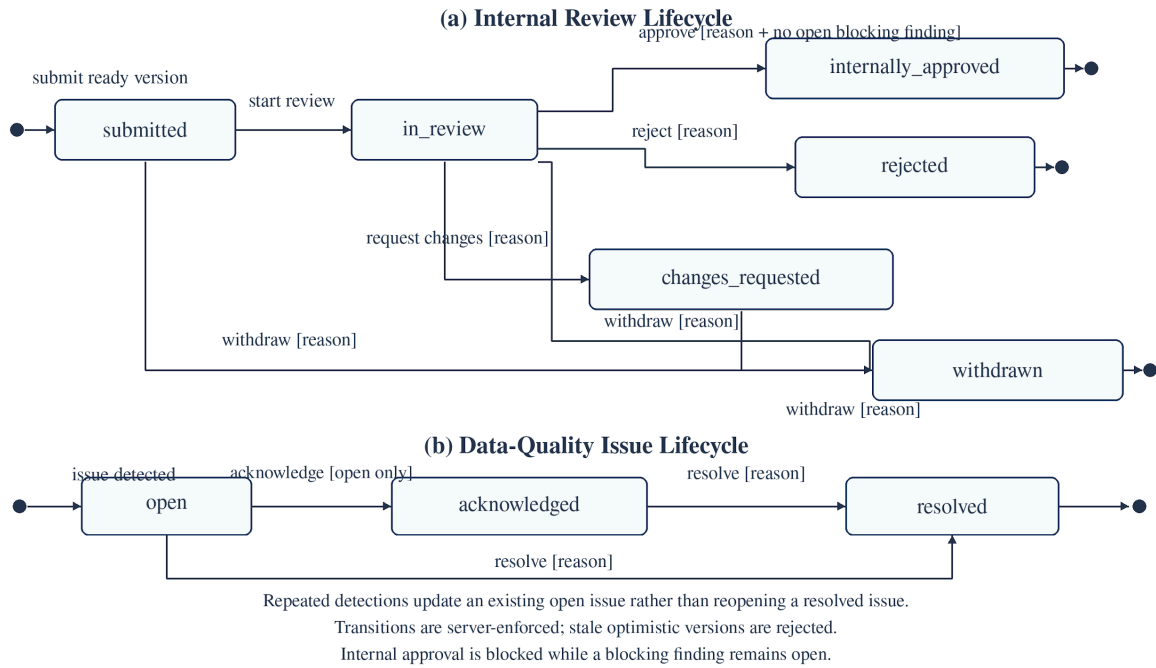


Fig 9(b): Persisted state transitions for internal review and data-quality issues.

5.4.2 Sequence Diagrams

The two sequences focus on the highest-risk persisted workflows: immutable calculation commit and version-pinned internal review. Participants and calls are named from the implemented Next.js/FastAPI/domain boundaries rather than conceptual placeholders.

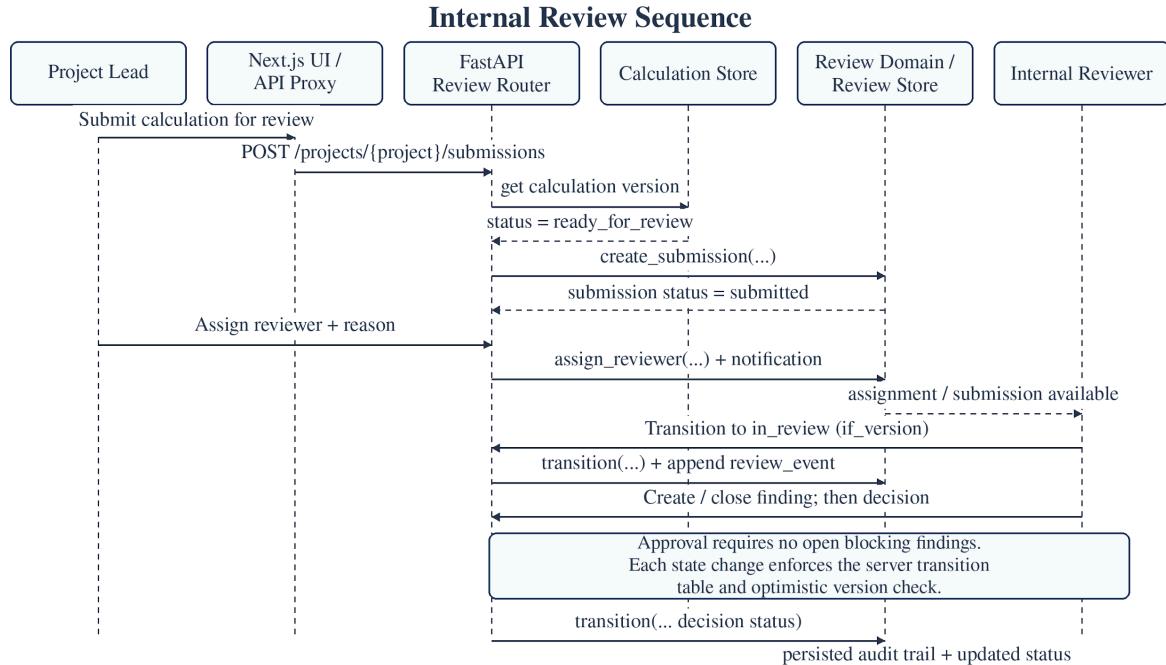


Fig 9(c): Internal review sequence from eligible calculation submission through reviewer assignment, guarded transitions, findings and the persisted review audit trail.

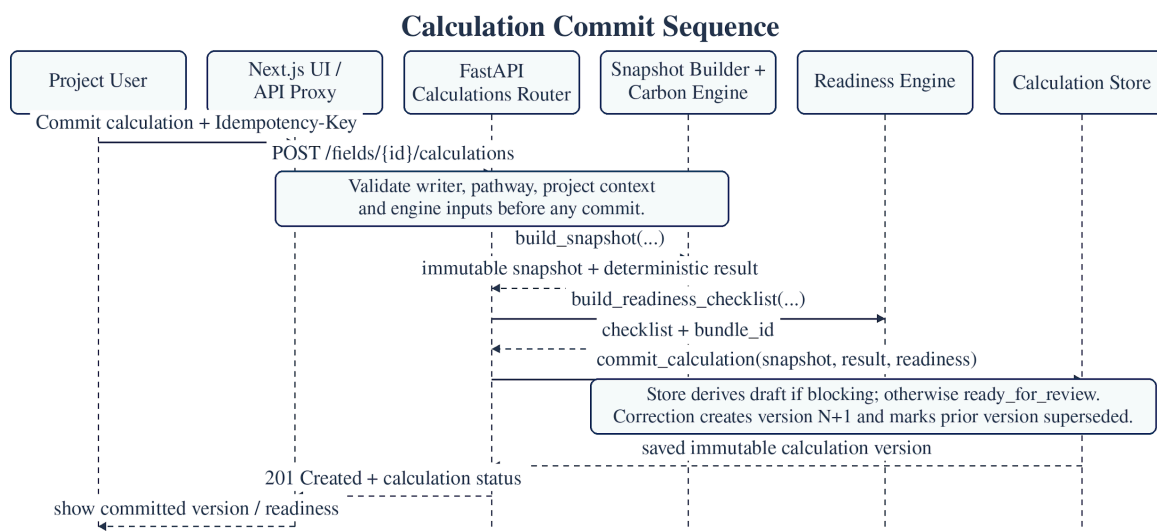


Fig 9(d): Calculation commit sequence from authenticated request through frozen snapshot, deterministic calculation, readiness evaluation and immutable version persistence.

5.5 Deployment Design

The deployment model separates the browser-facing frontend, API, transactional database/storage, worker, and external services. The documented production topology uses Vercel for the Next.js frontend and Render for the FastAPI backend; the repository also retains an optional Railway deployment configuration. A successful Vercel deployment status was observed, but the exact revision served by the submitted public URL, the exact live backend host, production database/storage state, and worker heartbeat were not independently established. A separate Python worker consumes the durable database-backed queue and calls Earth Engine or configured AI providers. The diagram therefore distinguishes documented/supported infrastructure from facts independently confirmed during report preparation.

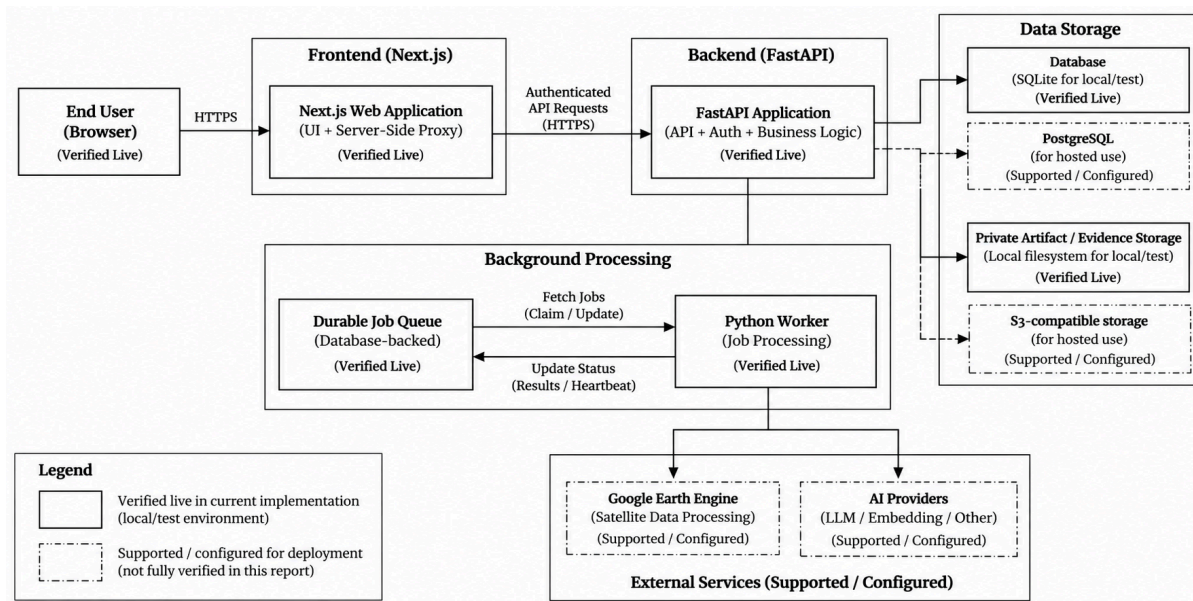


Fig 10: Terra-Audit Supported Deployment Topology. The public browser path, frontend, API, database/storage, background worker, and external Earth Engine/AI services are shown separately; labels distinguish documented/supported components from infrastructure that was not independently confirmed end to end.

5.6 Design Evolution and Alternatives

The final architecture reflects several deliberate refactorings from the midterm prototype. The Streamlit monolith was replaced by Next.js plus FastAPI so authentication, multi-user authorization, API contracts and responsive interface states could be separated from domain logic. Long-running signal and AI work moved from request-coupled execution to a durable database-backed queue with a separate worker. The old scalar/current-state calculation model evolved into versioned snapshots and readiness-aware calculation records to improve historical auditability. Methodology rules and carbon equations remain deterministic rather than being delegated to a language model; generative AI is constrained to draft explanation. SQLite remains useful for local/test work, while PostgreSQL and private object storage are supported for hosted operation. These choices increase implementation complexity but better match an evidence-centered MRV workflow than the original single-user prototype.

6. AI Engineering Design

6.1 Classical AWD ML Pipeline

The classical AWD ML pipeline applies to the rice signal path and remains experimental/supporting analytics rather than methodology-authoritative evidence:

- DatasetBuilder constructs training rows from cached rice time-series and uses threshold-gate labels; the label source is therefore not independent ground truth.
- Feature engineering builds a tabular matrix from satellite/time-series attributes while excluding the gate's own decision flags as direct predictor features.

- The shared training harness trains Random Forest (200 trees, class-balanced) and XGBoost (200 estimators, max depth 4) using field-grouped GroupKFold evaluation; at least two independent fields are required and each training fold must contain all label classes.
- Persisted model bundles store the model, classes, feature names, satellite processing version, and the field-grouped split strategy. Loading rejects stale processing versions so models are not silently applied to incompatible satellite features.
- Evaluation reports pooled out-of-fold threshold-agreement metrics such as precision/recall/F1, confusion matrices, feature importance, and one-vs-rest ROC data. The reporting code deliberately avoids presenting these values as independently verified irrigation accuracy.

6.2 Model Selection and Evaluation

Random Forest and XGBoost remain appropriate baselines because the current AWD features are a comparatively small structured tabular set rather than raw imagery. Both can model nonlinear feature interactions, are inexpensive to retrain, and expose feature importance. The final implementation evaluates them with the same field-grouped split strategy so model comparison is not inflated by placing observations from the same field in both training and validation folds.

Field grouping fixes an important evaluation-leakage problem from the earlier design, but it does not fix label provenance. Because the target labels are still generated by the threshold detector, model metrics measure how closely the models reproduce that detector. Independent water-level or irrigation observations are still required before the ML results can support a claim of real-world AWD event accuracy.

6.3 Managed Crop Models

The final system also contains managed crop-model workflows for exploratory crop classification/benchmarking. These models use reviewed crop-identity labels and held-out grouping, but they remain research features rather than carbon-accounting authority. Their presence broadens the AI workspace beyond AWD classification without changing the deterministic VM0051/VM0042 calculation boundary.

6.4 Generative AI Explanation Architecture

Generative AI is implemented as a bounded explanation workflow rather than as an accounting engine. An authenticated project request selects an allowed explanation action, after which the server builds a deterministic action-specific context packet from current project/evidence state and bundle-scoped methodology text. The request is served from cache when an equivalent validated explanation exists; otherwise a durable `ai_explain` job is queued and processed by the worker. Supported provider adapters include fake, self-hosted, Groq, and OpenAI.

The worker rechecks authorization, provider configuration, and evidence context before calling the provider. The structured response is then validated by server code and may receive at most one repair attempt. A final authorization/evidence check is performed before the explanation is persisted and shown as a draft requiring human review.

6.5 Prompt and Context Design

Prompting is therefore applicable in the final system, but the design is intentionally constrained by server-owned context. Packets contain the requested action, active methodology bundle, selected methodology chunks and linked corrections, evidence/calculation context, sentence identifiers, allowed requirement/record identifiers, hashes, and server-owned limitations/next actions. The model is not permitted to invent its own authoritative requirement identifiers or alter official calculation values. Retrieval is bundle-scoped so explanations are grounded in the methodology version/corrections selected by the application rather than in unrestricted general knowledge.

The output contract is structured rather than free-form only. Validation checks schema compliance, supplied sentence/requirement/record membership, citation presence, numeric/reference support, correction handling, prohibited claims, and server-owned actions/limitations. These checks improve structural reliability but do not prove that every cited sentence semantically entails the generated claim.

6.6 AI Authority Boundary and Evaluation Limitations

AI features are deliberately outside the accounting authority boundary. Random Forest/XGBoost and managed crop models are experimental analytics; generative providers produce draft explanations only. Deterministic code owns methodology readiness, carbon equations, persistence gates, and official numeric outputs. No AI component can turn a blocked requirement into compliant status, perform external VVB verification, or create registry-issued credits.

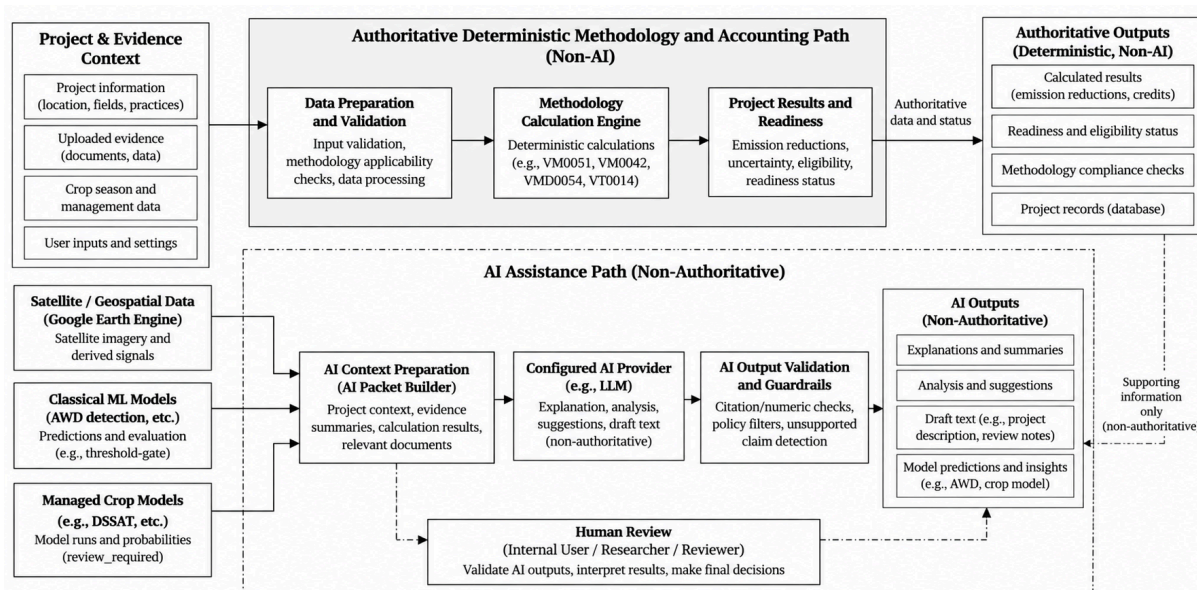


Fig 11: Terra-Audit AI Authority and Data Flow. Experimental AWD and managed-crop models provide analytics, while generative providers receive deterministic, bundle-scoped context through durable jobs and return validated human-review drafts. Deterministic server code remains authoritative for carbon calculations, readiness, evidence state, and validation gates.

The recorded software regression suite covers the AI code paths, but the dedicated 20-case generative-AI evaluation corpus remains incomplete because the canonical packet fixtures are missing. Likewise, no independent AWD ground-truth dataset has been completed. These gaps are reported as evaluation limitations rather than hidden behind software-test success.

7. Testing Strategy and Results

7.1 Testing Objectives

- Verify the deterministic VM0051 v1.1 and VM0042 v2.2 scoped accounting engines against their implemented factors, equations, golden values, edge cases, and hard numerical gates.
- Verify methodology/readiness behavior so unsupported, incomplete, stale, reviewer-dependent, or hard-blocking conditions remain explicit and cannot silently become satisfied or zero.
- Verify versioned calculation/snapshot behavior: later evidence, corrected seasons, recalculation, or supersession must create new state without silently rewriting the historical numerical inputs/results of an existing calculation version.
- Verify authentication, organization isolation, project-role checks, private evidence access, account recovery, and durable job/worker state transitions across protected backend workflows.
- Verify frontend contracts through automated tests, linting, TypeScript checking and a production build, and verify that backend changes do not break request/response schemas relied on by the Next.js application.
- Keep software verification distinct from scientific and external assurance: passing tests do not establish independent AWD field accuracy, production security certification, PostgreSQL migration correctness, live provider reliability, or VVB/Verra validation and issuance.

7.2 Test Strategy and Pass/Fail Criteria

Testing is layered so that a green arithmetic engine alone cannot hide a failure in authorization, evidence persistence, background processing, or frontend contracts. Unit and regression tests cover deterministic calculation logic and helpers; API/integration tests exercise authentication, tenant scoping, evidence, methodology, calculation, export, and job workflows against isolated databases; frontend tests, lint, TypeScript, and production build checks validate the web layer; and manual/operational checks cover deployment and external-service concerns that are not reproduced by the isolated suite.

A software verification item passes when the expected behavior completes without an unexpected failure in the recorded final verification. For the regression suite this means zero failed backend tests, zero failed frontend tests, a successful production build and TypeScript check, and no ESLint errors. A methodology or scientific claim has a higher bar: an automated test can prove that code follows a specified equation or gate, but it cannot prove methodology completeness, independent AWD event accuracy, production security certification, or external VVB/Verra acceptance.

7.2.1 Representative Direct API Endpoint Verification

The recorded backend suite contains direct FastAPI TestClient coverage for the representative routes below. Only endpoints with located HTTP-level test evidence are listed; lower-level domain tests are not relabelled as API verification.

API-01 — POST /auth/login: 200 + token for valid credentials; 401 for wrong/unknown credentials. Evidence: test_auth_endpoints.py.

API-02 — GET /auth/me: 200 with authenticated organization/role/email; 401 without a token. Evidence: test_auth_endpoints.py, test_registration.py.

API-03 — POST /fields: 201 for authorized admin; 403 viewer; 409 duplicate ID; 422 invalid field type. Evidence: test_fields_endpoints.py, test_field_type_guards.py.

API-04 — GET /fields/{field_id}: 200 for owning tenant; 404 for another organization. Evidence: test_fields_endpoints.py, test_field_type_guards.py.

API-05 — POST crop-season / GET evidence: 201 authorized create; 403 viewer; 422 invalid payload; 404 cross-tenant evidence access. Evidence: test_multicrop.py.

API-06 — POST monitoring-run / GET job: 202 durable job, pending before worker dispatch, done after execution, 404 cross-tenant. Evidence: test_multicrop.py.

API-07 — POST /fields/{field_id}/signal-runs: 200 on compatible local cache hit; 202 durable job on cache miss. Evidence: test_signal_run_cache_path.py.

API-08 — POST /fields/{field_id}/calculations: 201 with Idempotency-Key and valid ALM context; persists immutable calculation/snapshot state. Evidence: test_evidence_verification.py.

API-09 — GET /calculations/{calculation_id}/evidence/pdf: 200 application/pdf for owner, 404 cross-tenant, 403 unauthorized viewer; PDF remains snapshot-backed after live evidence changes. Evidence: test_evidence_verification.py.

API-10 — GET/POST methodology corrections: readable list; 403 viewer confirm; 200 admin confirm; 404 missing resource. Evidence: test_methodology_corrections.py.

API-11 — GET/POST /team/users: 403 non-admin; 200 admin list; 201 invite; 409 duplicate email; 422 invalid role. Evidence: test_team_router.py.

Review, batch, AI-provider and other behavior is also covered in the wider regression/integration suite, but it is not presented here as direct endpoint evidence unless a TestClient case was located. Overall regression status remains in Table 8.

7.2.2 Selected Detailed Test Outcomes

The cases below expand representative automated evidence into the SE801 outcome format. They are recorded outcomes from the final regression results; they were not rerun during final-report preparation.

TC-01 — VM0051 QA3 project-size hard gate.

Precondition/Input: CarbonAssetEngine with 2 AWD events, 120-day season, 500,000 ha and zero N input.

Steps: Execute `calculate_credits()` through the rice regression fixture.

Expected: The QA3 pathway must reject a result above the 60,000 tCO₂e/year applicability gate; no issuable/internal final quantity may be returned.

Recorded observed outcome: `qa3_pathway_valid=False`, `final_issuance=None`, and the block reason identifies the 60,000 threshold.

Status/Evidence: RECORDED PASS — `tests/test_carbon_calculator.py`.

TC-02 — VM0042 calculation without integrated leakage evidence.

Precondition/Input: Valid ALM practice/SOC fixtures for 5 ha, but no frozen `leakage_result`.

Steps: Execute `AlmCarbonEngine.calculate_credits()`.

Expected: Missing production/leakage evidence must block rather than silently substitute zero leakage.

Recorded observed outcome: `production_decline_leakage_blocked=True`, `final_issuance=None`, the block reason requests a scoped leakage assessment, and `result_is_issuable()` is false.

Status/Evidence: RECORDED PASS — `tests/test_carbon_calculator_alm.py`.

TC-03 — Cross-organization field isolation.

Precondition/Input: A field exists in organization `testorg`; request is authenticated as an administrator of `otherorg`.

Steps: GET `/fields`, then GET `/fields/{field_id}`.

Expected: The other organization cannot enumerate or directly retrieve the field.

Recorded observed outcome: the other-organization field list is empty and direct retrieval returns HTTP 404.

Status/Evidence: RECORDED PASS — tests/backend/test_fields_endpoints.py.

TC-04 — Snapshot-backed ALM PDF remains frozen after live evidence changes.

Precondition/Input: Commit an evidence-linked ALM calculation with an Idempotency-Key and retain its calculation snapshot.

Steps: Verify cross-tenant/viewer access rejection; mutate current field metadata and baseline practice after commit; request GET /calculations/{calculation_id}/evidence/pdf.

Expected: Unauthorized access is rejected, while the owner receives a PDF generated from the committed snapshot/result rather than rereading changed live evidence.

Recorded observed outcome: cross-tenant=404, viewer=403, owner=200 application/pdf; captured field, practices, livestock, result and readiness match the committed calculation snapshot even after live records change.

Status/Evidence: RECORDED PASS — tests/backend/test_evidence_verification.py.

TC-05 — Durable monitoring job lifecycle and tenant scope.

Precondition/Input: Authorized crop season with a deterministic monitoring fixture.

Steps: POST a monitoring run; inspect the public job before dispatch; let a registered worker claim and execute the job; inspect the final job and evidence; query the job from another organization.

Expected: Request returns 202 with pending state, worker claim changes it to running, completion changes it to done, frozen evidence remains stable, and another organization cannot read the job.

Recorded observed outcome: pending → running → done is asserted; cross-organization job access returns 404; previously stored evidence remains unchanged after current field metadata is edited.

Status/Evidence: RECORDED PASS — tests/backend/test_multicrop.py.

TC-06 — Generative-AI output validation rejects unsupported authority claims.

Precondition/Input: Deterministic explanation packet allowing one evidence sentence and one methodology requirement.

Steps: Validate responses containing unsupported values/units, invented records or requirements, or prohibited claims such as compliant, approved, certified or eligible for issuance; exercise the one-retry generation path.

Expected: Unsupported/prohibited output must not be persisted as a successful explanation; one corrected retry may pass, while repeated invalid output must fail.

Recorded observed outcome: invalid claims raise `ExplanationValidationError`; a bad first response followed by supported content succeeds with `retry_count=1`; two invalid attempts raise an error and no partial result is returned.

Status/Evidence: RECORDED PASS — `tests/test_ai_validate.py`.

7.3 Final Verification Results

Table 8: Recorded Final Verification Outcomes

Verification Area	Command / Evidence	Scope	Pass Criterion	Final Status
Backend/core regression	<code>python -m pytest -q</code>	Full current backend/core suite	0 failed; 0 skipped unless explicitly justified	RECORDED LOCAL — 177 passed, 0 failed, 0 skipped; 21 warnings; 119.00 s (not rerun in this audit)
Frontend tests	<code>npm test</code>	Current frontend test runner	0 failed	RECORDED LOCAL — 4 passed, 0 failed, 0 skipped (not rerun in this audit)
Production build	<code>npm run build</code>	Next.js production build	Exit 0 and completed build	RECORDED LOCAL — build exited successfully (not rerun in this audit)
TypeScript	<code>npm run typecheck</code> after build	Frontend type checking with generated Next types available	Exit 0	RECORDED LOCAL — TypeScript check exited successfully (not rerun in this audit)
ESLint	<code>npm run lint</code>	Frontend lint rules	0 errors	RECORDED LOCAL — 0 errors, 5 warnings (not rerun in this audit)
Repository state at recorded verification	<code>git status</code> ; branch/HEAD check	Public repository and working-tree state	No staged, modified, or untracked files	RECORDED LOCAL — the working tree was reported clean when verification was recorded; no later rerun is claimed
VM0051/rice accounting	Included in backend regression suite	Rice equations, uncertainty/N2O logic, size and edge gates	No current regression	Covered by the recorded 177-test run; not

Verification Area	Command / Evidence	Scope	Pass Criterion	Final Status
				independently rerun here
VM0042/ALM + leakage	Included in backend regression suite	SOC/non-SOC terms, leakage adapter, hard gates, versioned commit path	No current regression	Covered by the recorded 177-test run; not independently rerun here
Methodology/readiness	Included in backend regression suite	Bundles, corrections, applicability/readiness and blocking behavior	Unsupported/blocking states remain explicit	Covered by the recorded 177-test run; not independently rerun here
Evidence/snapshot/export	Included in backend regression suite	Evidence mapping, frozen snapshots, historical exports/idempotency	Historical payloads remain stable under tested changes	Covered by the recorded 177-test run; not independently rerun here
Authentication/tenant access	Included in backend regression suite	Account, registration, organization/team/field guards	Unauthorized/cross-tenant access rejected in covered paths	Covered by the recorded 177-test run; not independently rerun here
Durable jobs/worker behavior	Included in backend regression suite	Queue state, job handlers, retry/cancellation-related behavior	No synchronous-legacy regression	Covered by the recorded 177-test run; not independently rerun here
AI structural validation	Included in backend regression suite	AI packets, allowed references/citations/numeric support and validator behavior	Invalid structured output rejected in covered cases	Covered by the recorded 177-test run; not independently rerun here
CI type-generation ordering	Current CI config compared with local final run order	Next-generated global types used by frontend	CI should generate types before standalone typecheck	OPEN CONFIGURATION GAP — recorded local order test → build → lint → typecheck passes; CI ordering was not changed
Generative-AI canonical evaluation	AI evaluation manifest/runner	20 frozen canonical packet cases	All canonical fixtures present and evaluation executes	NOT EXECUTED — canonical fixture set remains incomplete

Verification Area	Command / Evidence	Scope	Pass Criterion	Final Status
Browser E2E / accessibility	No final browser-E2E run recorded	Full user journeys, keyboard/accessibility and responsive behavior	No critical end-to-end defects	NOT EXECUTED as a formal automated suite
Live external-service reliability	No comprehensive final live-service fault campaign	Earth Engine, AI/email providers, shared storage and worker availability	Graceful recovery under live failure	NOT FULLY VERIFIED
PostgreSQL migration/run time	No final remote migration/write verification	Current PostgreSQL schema and existing-data upgrade path	Successful current-schema migration and integrity verification	NOT INDEPENDENTLY VERIFIED
Independent AWD ground truth	No completed field-validation campaign	Satellite/threshold/ML detections vs independent water-level/irrigation observations	Event-level empirical accuracy metrics	NOT EXECUTED — software tests do not establish scientific validation

Recorded local verification reports: backend/core 177 passed, 0 failed and 0 skipped (21 warnings; 119.00 seconds); frontend npm tests recorded 4 passed; production build and TypeScript passed after generated Next.js types were available; ESLint recorded 0 errors and 5 warnings. The local suites were not rerun during final-report preparation. No GitHub Actions workflow run was found for the reviewed repository state, and the checked workflow configuration does not support claiming a CI pass for these recorded local results. Scientific, browser-E2E, PostgreSQL, provider, and external-assurance checks remain separate limitations.

7.4 Risks and Contingencies

Earth Engine or Sentinel availability — Signal requests must surface empty-data, authentication, network, or API failures as explicit job/error states. Cached compatible observations may reduce repeat dependency but do not replace provenance or live-service testing.

Worker unavailable or stale — Durable jobs remain persisted rather than disappearing with the API request. Operations views and heartbeat state are used to detect stale workers; supported retry/cancel paths are preferred over pretending completion.

AI provider unavailable or invalid output — Explanation requests remain optional. Cached validated explanations may be reused; otherwise provider/configuration errors, packet-size failures, or structured-validation failures remain visible and no authoritative readiness/calculation state is written.

Database/storage failure — SQLite is suitable for local/test operation and PostgreSQL is supported for hosted operation, but current PostgreSQL migration/restore behavior is not fully verified. Private evidence/model artifacts require authenticated storage access and a backup/restore strategy.

Insufficient satellite observations or weak ground truth — The system can fall back to explicit manual calculation inputs where allowed, but the absence of observations cannot be converted into a scientific AWD claim. Independent field validation remains a separate research requirement.

Methodology incompleteness — Unsupported or unresolved methodology/program requirements remain disclosed as partial, unsupported, reviewer-dependent, or blocking. The contingency is not to weaken a gate for convenience; it is to keep the calculation as draft/limited until the missing requirement is addressed by supported evidence or an external expert/VVB process.

7.5 Requirements Traceability Matrix

Compact mapping from each final functional requirement to the current use case/goal, implementing component, verification evidence and final status. “Pass within final suite” refers to the recorded 177-test backend/core regression result; partial statuses identify implementation or assurance boundaries that remain even when the software tests are green.

Table 9: Functional Requirements Traceability Matrix

Req.	Use Case / Goal	Component	Verification	Status
FR1	Account access / tenant identity	src/accounts; backend/security.py; auth/account-access routers	Authentication/account- access tests	Automated — pass within final suite
FR2	Project & Field Management	src/projects; field/project/team routers; persistence	Project/team/field regression tests	Automated — pass within final suite
FR3	Signal Analytics	src/signals; src/jobs; signal router/worker handler	Signal/cache/job regression tests	Automated — pass within final suite
FR4	Signal Analytics / AWD research	processing.py; threshold_gate.py; src/ai/ml	Signal and ML regression tests	Implemented; software-tested; scientific validation pending
FR5	Evidence & Season Management	src/evidence/monitorin g.py; attachments/monitoring routers	Evidence/monitoring regression tests	Automated — pass within final suite
FR6	ALM Evidence	src/evidence/soil.py; production.py; ALM schedules	Evidence/ALM regression tests	Automated — pass within final suite
FR7	Carbon Calculation — rice	src/carbon/rice.py; calculation/carbon routers	Rice carbon regression tests	Automated — pass within final suite
FR8	Carbon Calculation — ALM	src/carbon/alm.py; leakage_vmd0054.py; snapshot.py	ALM/leakage regression tests	Automated — pass within final suite
FR9	AI Assistance & Research	src/ai/ml; managed-model modules	ML/model regression tests and code review	Implemented/experi mental; independent accuracy pending
FR10	AI Explanation	src/ai/packets.py; providers.py; validate.py; AI job handlers	AI packet/validator regression tests	Implemented/partia l; canonical 20-case evaluation pending
FR11	Reporting & Export	src/reporting; backend/routers/export .py	Evidence/snapshot/export regression tests	Implemented/partia l; legacy provenance limits remain

Req.	Use Case / Goal	Component	Verification	Status
FR12	History & Portfolio	calculations/version chain; project/history/portfolio views	Calculation/history/portfolio regression coverage	Implemented; final suite green
FR13	Methodology Readiness	src/methodology/registry.py; library/corrections/readiness.py	Methodology/readiness regression tests	Implemented/partial methodology coverage
FR14	Calculation Versioning	src/carbon/snapshot.py; calculations.py; idempotency	Snapshot/version/idempotency tests	Automated — pass within final suite
FR15	Internal Review	src/projects/reviews.py; reviews router/UI	Review-route/domain regression tests	Implemented; entry still depends on readiness blockers
FR16	Portfolio / Operations	portfolio/product-ops/ monitoring-ops views and routers	Backend regression suite + operational review	Implemented/partial
FR17	Authorization & editable metadata	backend/access.py; deps.py; field/project update paths	Tenant/auth/team/field tests	Implemented/partial; project enforcement not uniform everywhere
FR18	Durable Jobs	src/jobs/queue.py; worker.py; job handlers	Job/multicrop/background-work tests	Automated — pass within final suite

8. Final Implementation

This chapter summarizes the final implemented software rather than reporting midterm progress. Status labels distinguish implemented capability from partial methodology coverage, experimental research features, and validation/operational gaps. A green software test suite does not by itself convert a partial or experimental feature into a fully validated MRV method.

8.1 Final Feature Status

Status is reported conservatively. “Implemented” means the software capability exists and is covered by the current architecture; “Partial” means important scope, provenance, methodology, authorization, or operational limits remain; “Experimental” identifies research features that must not be presented as validated accounting evidence.

Table 10: Final Implementation by Subsystem

Module / Feature	Status	Current Evidence	Remaining Work / Boundary
Next.js frontend	Implemented	Authenticated pages/components, responsive routes, API proxy; frontend tests/build/typecheck pass.	Formal browser-E2E/accessibility suite and final-state screenshot capture remain.
FastAPI backend/API	Implemented	Domain routers/schemas, auth dependencies, project/evidence/calculation/job/AI/export APIs; backend suite green.	Continue endpoint-level authorization hardening and API/deployment observability.

Module / Feature	Status	Current Evidence	Remaining Work / Boundary
Authentication & organizations	Implemented	bcrypt/JWT/token-version/account-recovery flows and server-derived organization scoping.	Login throttling and some production account-policy hardening remain.
Project roles & memberships	Partial	Project lead/contributor/viewer roles, membership events, project/farm-field relationships and access helpers exist.	Project-membership enforcement is not uniform across every field/calculation endpoint.
Evidence / seasons / attachments	Implemented	Crop-season corrections/history, observations, practice events, reviews, monitoring, QA issues and private evidence attachments.	Broader browser acceptance and retention/deletion edge cases remain.
Soil / SOC / lab / custody	Partial	Sampling plans, strata, samples, lab results, custody events and reviewed SOC mapping are implemented.	Full sampling/statistical methodology compliance and multi-stratum accounting remain incomplete.
Sentinel-1 signal processing	Implemented	Earth Engine acquisition, cache/versioning, smoothing, threshold/phenology processing and durable signal jobs exist.	Full acquisition provenance, orbit/incidence-angle/rainfall controls and live-service recovery validation remain.
AWD threshold & classical ML	Experimental	Threshold detector plus field-grouped RF/XGBoost evaluation and versioned feature/model metadata are implemented.	No independent water-level/irrigation ground truth; metrics remain model-versus-threshold agreement.
VM0051 rice accounting	Partial	Scoped VM0051 v1.1 QA3/default-factor calculations and gates are deterministic and regression-tested.	Rice leakage, baseline fertilizer-change N20, burning and several monitoring/safeguard requirements remain unsupported/partial.
VM0042 ALM + leakage	Partial	Scoped VM0042 v2.2 + corrections, supported SOC/non-SOC terms and restricted VMD0054 v1.1 leakage adapter are implemented.	QA1/modeling, full DSM/sampling, multi-field leakage, mitigation and cumulative sequential-period reconciliation remain incomplete.
Methodology registry & readiness	Partial	Documents, bundles, corrections, applicability and deterministic readiness/fingerprints are implemented.	common.additionality remains unsupported, blocking and non-overridable; methodology coverage is not complete.
Calculation snapshots/versioning	Implemented	Frozen calculation inputs/results/readiness context, idempotency and supersession/version chains are implemented.	No DB-level WORM/immutability trigger; some provenance selections are broader in backend than current UI.
Durable jobs & worker	Implemented	Database-backed queue, worker heartbeat, job handlers, retry/cancel states and operational views exist.	Live production worker/heartbeat and failure-recovery drills remain incompletely verified.
Internal review & exports	Partial	Version-pinned internal review, findings/events, versioned calculation JSON, snapshot-backed ALM PDF, and	Rice/legacy export provenance is weaker than ALM snapshot export;

Module / Feature	Status	Current Evidence	Remaining Work / Boundary
		legacy verification PDF/JSON/CSV paths are implemented.	internal review is not VVB verification.
AI explanations & managed models	Partial / Experimental	Provider adapters, deterministic packets, validation, durable ai_explain jobs and managed crop-model workflows exist.	20-case generative-AI evaluation fixtures are incomplete; semantic entailment and scientific accuracy are not proven.
Recorded software verification	Recorded; not independently rerun	Recorded verification reports backend 177/0/0 plus passing frontend tests/build/typecheck and lint with 0 errors/5 warnings. A successful Vercel deployment status was also observed; the local suites were not rerun during final-report preparation.	Scientific field validation, browser-E2E, PostgreSQL live migration/runtime and external assurance remain separate.

Reproducible worked calculation checks

Rice AWD fixture — Inputs: 2 AWD events, 120-day season, 1.0 ha, 100 kg N/ha, short pre-season water regime, and the default 5 t/ha straw amendment in both scenarios. The engine gives $SC_o = (1+5)^{0.59} = 2.878122$; baseline $CH_4 = 293.568470$ kg and project $CH_4 = 161.462659$ kg, so avoided $CH_4 = 132.105812$ kg = 3.698963 tCO₂e. The QA3 15% deduction is 0.554844 tCO₂e, leaving 3.144118 tCO₂e; the irrigation N₂O term is 0.083210 tCO₂e. Net deterministic estimate = 3.060908 tCO₂e, below the 60,000 tCO₂e/year QA3 gate. This is a code-level worked fixture, not an issued VCU.

Cropland ALM fixture — Inputs follow the fixed 5 ha, one-year regression scenario: baseline synthetic N 120 kg/ha, residue burning 500 kg/ha and diesel 40 L/ha; project synthetic N 80 kg/ha, organic N 20 kg/ha, no residue burning and diesel 25 L/ha; SOC cells are project start [30,32,29], project final [34,35,33], control start [30,31,29], control final [30,31,30]. The mean project-minus-control SOC change is 3.333333 tCO₂e/ha; the independently reconstructed uncertainty is 16.115318%. The engine regression fixture gives ER=1.073643 tCO₂e and uncertainty-adjusted CR=13.980780 tCO₂e; with a 15% non-permanence deduction on the removal term, the deterministic estimate is 12.957306 tCO₂e. The fixture still discloses the separate other-leakage limitation and does not imply external verification or issuance.

8.2 Remaining Engineering and Validation Gaps

Table 11: Remaining Engineering and Validation Gaps

Current Gap / Risk	Next Response
No independent AWD ground truth; threshold and RF/XGBoost labels are not field observations.	Design a Bangladesh/South Asia validation campaign using water-level tubes, irrigation logs or partner-verified observations and report event-level precision/recall, timing error and false drydown rate.
Methodology completeness and additionality readiness remain incomplete.	Keep unsupported/blocking requirements explicit; resolve the additionality workflow with a true expert/VVB-style determination path rather than weakening the gate; continue methodology-by-methodology applicability review.
Rice signal/calculation/export provenance is incomplete on legacy paths.	Freeze exact saved signal/evidence context for each committed rice calculation and historical export, including available scene/configuration/boundary/version metadata and software revision.

Current Gap / Risk	Next Response
Authorization and data-integrity enforcement are not uniform at the database/endpoint level.	Extend project-membership checks to remaining field/calculation routes; add stronger login/shared throttling, content inspection, cleanup checks, and consider database FK/RLS controls where appropriate.
Hosted PostgreSQL, shared storage, worker availability, backup/restore and deployed revision are not fully verified end to end.	Run a controlled production/staging operational drill covering current-schema PostgreSQL migration, worker heartbeat/restart, shared object storage, backup restoration, deployed revision identification and external-service recovery.
Generative-AI quality evaluation is incomplete despite structural validation.	Create the 20 frozen canonical context-packet fixtures, run provider acceptance evaluation, and add human semantic review; keep outputs as drafts regardless of provider score.
Scientific and external assurance remain separate from software regression results.	Do not equate 177 passing tests with scientific AWD validation, VVB verification, Verra approval or issued VCUs; keep these claims explicitly separated in UI, exports and the final report.

9. Project Timeline and Final Status

9.1 Planned Schedule and Final Verification Point

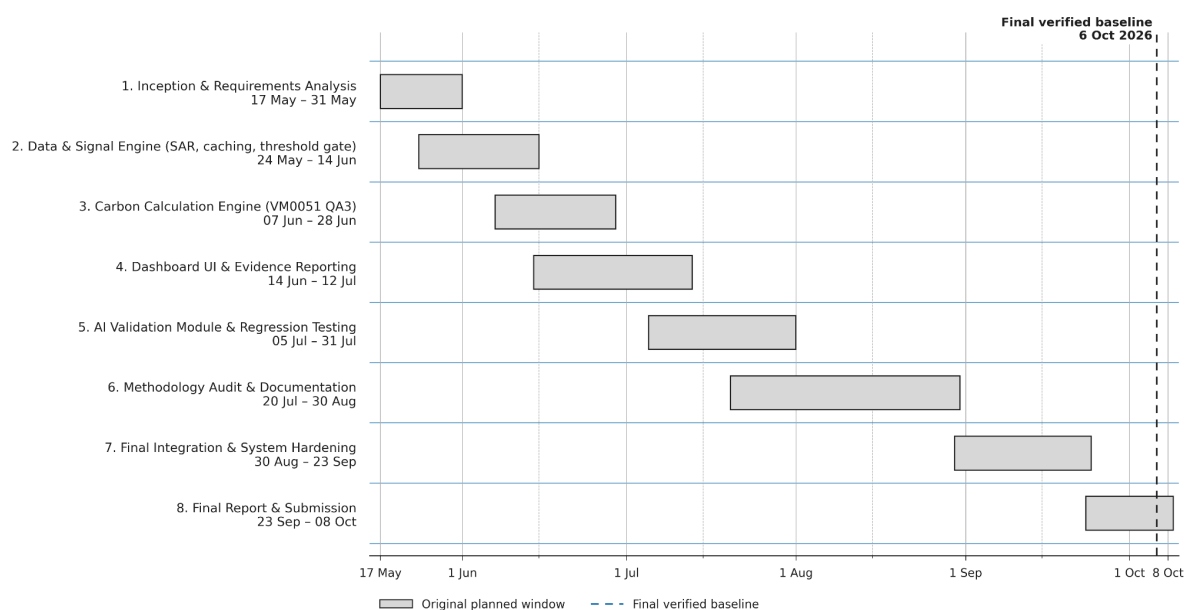


Fig 12: Project Timeline (Gantt Chart). The original planned schedule is shown with the final verification point on 6 October 2026. The bars preserve the semester plan; the verification marker shows that implementation, testing, and documentation continued beyond the original submission window.

Table 12: Project Phases, Planned Windows, and Final Recorded Status

Phase	Original Planned Window	Final Project Status (6 Oct 2026)
1. Inception & Requirements Analysis	17 May – 31 May	Complete; requirements evolved from single-user prototype scope to authenticated multi-user evidence/MRV workflows.

Phase	Original Planned Window	Final Project Status (6 Oct 2026)
2. Data & Signal Engine (SAR, caching, threshold gate)	24 May – 14 Jun	Implemented; durable signal jobs, cache/versioning and experimental ML support added. Scientific/provenance hardening remains.
3. Carbon Calculation Engine (VM0051 QA3)	07 Jun – 28 Jun	Implemented as scoped VM0051 v1.1 deterministic engine; methodology completeness remains partial.
4. Dashboard UI & Evidence Reporting	14 Jun – 12 Jul	Original Streamlit UI replaced by Next.js + FastAPI; review/history/export workflows expanded. Legacy provenance limits remain.
5. AI Validation Module & Regression Testing	05 Jul – 31 Jul	Classical AWD ML and managed models implemented; field-grouped evaluation added. Independent AWD ground truth and canonical GenAI evaluation remain incomplete.
6. Methodology Audit & Documentation	20 Jul – 09 Aug	Methodology documents/bundles/corrections/readiness implemented and documented; unsupported/blocking requirements remain explicit.
7. Final Integration & System Hardening	09 Aug – 23 Aug	Substantial implementation completed: auth/orgs/roles, durable jobs, evidence snapshots, internal review and security controls; operational hardening remains partial.
8. Final Report & Submission	23 Aug – 04 Sep	Final report documentation and verification were completed for the submitted system; the submission date remains author/course controlled.

By the final reporting point, Terra-Audit had progressed substantially beyond the midterm prototype: the active system is a Next.js/FastAPI multi-user application with authenticated organizations/projects, structured evidence, durable jobs, methodology/readiness logic, versioned calculations, internal review, exports, classical/managed ML and bounded generative-AI explanations. The recorded software regression result is 177 backend/core tests passed with no failures or skips, with frontend tests, production build and TypeScript checks also passing. This software maturity does not remove the scientific, methodology, provenance or operational limitations documented elsewhere in this report.

Remaining final-project limitations / follow-up work:

- Complete independent AWD ground-truth validation and quantify event-level error against field observations.
- Finish methodology/provenance hardening for unsupported requirements and legacy rice/export paths.
- Complete the 20-case generative-AI evaluation corpus and add semantic human-review evaluation.

- Verify the current PostgreSQL migration/runtime, shared storage, worker recovery, backup restoration and deployed-revision traceability in a controlled environment.
- Extend authorization and database-integrity hardening where project membership, foreign keys/RLS, throttling, and file-content inspection remain incomplete.
- Keep VVB/Verra validation, verification and registry issuance outside the application claim boundary until external processes actually occur.
- Capture the final demonstration, interface screenshots and report/PDF visual QA before submission.

10. Interface Design

The final interface is a responsive Next.js web application centered on authenticated project context, explicit permissions/readiness, durable-job status, versioned calculation history and internal review. Queued, running, blocked, failed, review-required and completed states remain visibly distinct rather than being collapsed into a generic success/failure message.

10.1 Interaction Objects and States

Primary interface objects are accounts/sessions; projects, members and fields; crop/evidence/attachment workspaces; signal analytics and durable jobs; calculation/readiness/history views; internal-review records; AI/ML workspaces; methodology/setup screens; and operational worker/queue views. Actions are role-aware, while server-side authorization and deterministic accounting/readiness remain authoritative.

10.2 Primary User Workflows

The core user journey is: select project/field → build or review evidence → run rice signal analytics when applicable → preview and save a versioned calculation → inspect readiness → submit an eligible version to internal review → export an available artifact. Researchers use signal/ML/AI workspaces; administrators manage methodology sources, provider permissions and worker/job state.

10.2.1 UI Event-to-State Transition Summary

Fig 9 contains the persisted state machines; the summary below maps the most important visible user events to those server-owned states.

- Monitoring/job lifecycle — Accepting a durable request exposes pending; worker claim changes it to running; completion changes it to done; unrecoverable failure changes it to error. Pending jobs may cancel immediately, while running jobs move through cancel_requested before confirmed cancellation. Retrying failed batch work creates new pending job rows and preserves the failed history.
- Quality/review actions — Data-quality issues move open → acknowledged/resolved with actor/time/reason. A committed calculation becomes draft or ready_for_review; corrections create a new version and supersede the prior one. Only ready_for_review versions may enter internal review, whose server transition table governs submitted, in_review, changes_requested, internally_approved, rejected and withdrawn states.
- AI explanation — A compatible validated cache entry may return immediately; otherwise a durable AI job exposes pending/running/done/error/cancelled. Only done

exposes the validated draft, which remains human-review content and cannot change readiness or accounting state.

10.3 Responsive, Validation and Accessibility Design

The frontend uses shared labeled controls, reusable validation messages, narrow-screen scrolling/containment, text-plus-color state cues, standardized date/number formatting and keyboard-aware shared controls. These improve usability across desktop/tablet/mobile layouts, but no dedicated final browser-E2E or formal accessibility suite was independently executed.

10.4 Interface Evidence Boundary

Figures 13–17 document representative visible states of the current web interface using clearly labelled synthetic demonstration records: project navigation, monitoring, calculation readiness/input, review-related views and the AI workspace. Captions claim only what is visible; the screenshots do not prove unshown execution paths or constitute methodology, scientific, security or registry validation.

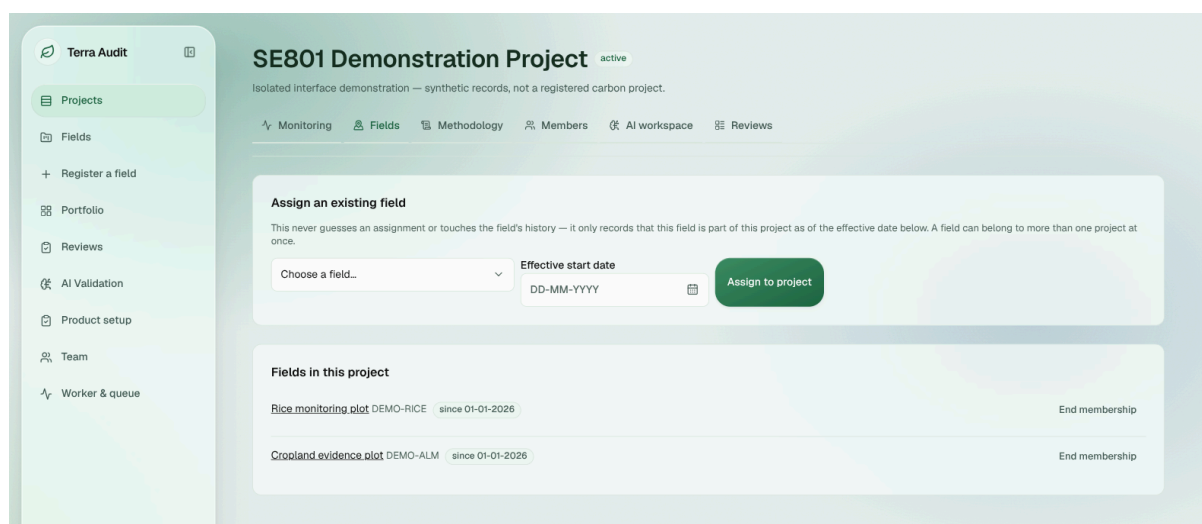


Fig 13: Application Shell and Project Navigation. Authenticated synthetic demonstration project showing the application sidebar, project tabs, field-assignment controls, and two demonstration fields.

Terra Audit SE801 Demonstration Project active

Isolated interface demonstration — synthetic records, not a registered carbon project.

Monitoring Fields Methodology Members AI workspace Reviews

FIELD-SEASONS 2 **COVERAGE READY** 0/2 **OPEN ISSUES** 0 **BATCHES** 1

Field-seasons Filter by crop All statuses

Field	Season	Crops	Latest status	Source	Last run	Issues
☐ Rice monitoring plot DEMO-RICE	Boro 2026	rice	no run yet	—	—	—
☐ Cropland evidence plot DEMO-ALM	Wheat 2026	wheat	no run yet	—	—	—

☐ Force refresh (skip reuse) Start monitoring (0)

Batches

running 2 Job(s) - 06-10-2026 20:30:39 GMT+6

Progress: pending: 2

Cancel remaining Retry failed

pending cc56a351
pending 2e97854e

Data-quality issues Open

Provisional engineering screens, not validated accuracy or management-practice compliance claims.
No issues.

Fig 14: Monitoring Batch State. Synthetic demonstration view showing one running batch with two pending child jobs; the field-season rows show “no run yet,” and the data-quality section currently reports no issues.

(a)

Readiness checklist

This reflects what this implementation can check automatically, plus any recorded expert determinations — it is not a certification of full methodology compliance.

needs review	Explain common.methodology_applicability Declared crop(s) [rice] are within pathway 'vm0051_rice_awd's usual scope per crop taxonomy alone — this is an indicative signal, not a full applicability determination. Actual land-use conditions (e.g. water regime, drainage, prior land use) must still be confirmed by a reviewer before this can be marked satisfied. Required evidence: Field type (immutable at registration) plus the crop(s) declared in linked crop seasons must be within the selected pathway's documented scope.
satisfied	common.monitoring_period_coverage Linked crop seasons cover the requested monitoring period. Required evidence: Crop-season start/end dates spanning the monitoring period.
needs review	Explain common.evidence_review_status 1 of 1 field observations are not yet accepted by an independent reviewer. Required evidence: Each field observation accepted by a team member other than its author.
missing	Explain vm0051.required_measurement_inputs AWD event count and season length are the engine's required satellite-derived inputs. VM0051: Improved Management in Rice Production Systems v1.1 §8.2.3 Eq. 6 Required evidence: AWD event count and season length from signal analytics or manual entry. Cannot be manually overridden.
needs review	Explain vm0051.qa3_pathway_project_size Run a calculation preview to evaluate the QA3 project-size gate. VM0051: Improved Management in Rice Production Systems v1.1 §8.6.3 Required evidence: Computed from the calculation result; not evidence supplied by a user. Cannot be manually overridden.
unsupported	Explain vm0051.leakage_assessment Not implemented — no evidence can satisfy this in the current codebase. VM0051: Improved Management in Rice Production Systems v1.1 §8.4 Required evidence: Not implemented — no evidence can satisfy this in the current codebase. Implementation: not implemented by this system Cannot be manually overridden.

(b)

Engine inputs

Manual evidence entry for this monitoring period. No measurements are assumed. Use zero only when the evidence records zero; amendment rates may be zero when no material was applied.

Use saved analytics

Only runs matching this field, registered area and both monitoring dates are offered. Confirm that the boundary and selected crop seasons match: older signal jobs do not freeze those versions. Nitrogen, water regime and amendments remain manual.

No compatible run among recent completed analytics. Run analytics for 01-01-2026 to 31-05-2026, or enter documented values manually.

Keep values as manual entry

AWD events (evidence-backed)	Season length (days)
N input (kg N/ha)	Pre-season water regime
	Select documented water regime
Baseline amendment type	Baseline amendment rate (t/ha)
Select documented amendment	
Project amendment type	Project amendment rate (t/ha)
Select documented amendment	

Field area used is always this field's registered area_ha (1.14 ha) — it is frozen from the field record, not re-entered here.

Correct an existing calculation (optional)

New calculation chain

☐ I have checked these values against evidence for the selected period.

Preview calculation Commit (freeze evidence)

Fig 15: Rice Calculation Readiness and Inputs. The captured synthetic state shows (a) readiness items marked needs review, satisfied, missing, or unsupported and (b) the evidence-backed engine-input form with Preview calculation and Commit (freeze evidence) actions. No numerical calculation result is visible in this screenshot.

(a)

Reviews

Internal review only — this is not external verification or registry issuance.

Notifications
 No notifications.

My reviews
 Nothing assigned to you
 Submissions assigned to you for review will appear here.

Project review queue
 SE801 Demonstration Project ▾

All statuses ▴
 All statuses ✓
 submitted
 In review
 changes requested
 internally approved
 rejected
 withdrawn

 No submissions
 No submissions have been submitted for review in this project yet.

(b)

Record an evidence review

Project leads and administrators can decide reviewable requirements. Decisions apply to the selected project, dates and current evidence. Changed leakage inputs or production records require a new review.

common.methodology_applicability ▾
 Further review needed ▾
 Demo evidence requires independent confirmation of water regime and prior land use. No applicability approval has been granted.

Record review

Calculation history
 draft v1 06-10-2026 20:30:39 GMT+6 Estimate: 1.4627 tCO2e
 Explain why blocked What changed? Download JSON

Fig 16: Review-Related Interface States. The captured synthetic views show (a) the internal Reviews page with no assigned or submitted reviews and its status filter and (b) an evidence-review form plus a draft calculation estimate. The screenshot does not show an active review submission, finding, comment, or approval event.

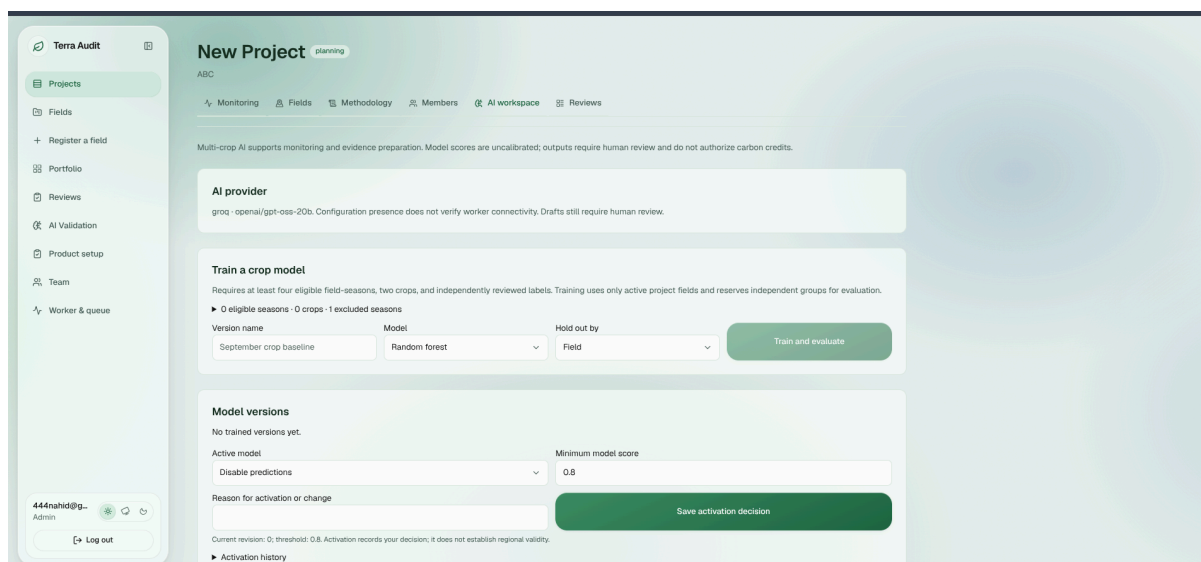


Fig 17: AI Workspace — Crop-Model Controls. The captured interface shows the configured provider label, crop-model eligibility count, training/evaluation controls, no trained model versions, and predictions disabled. It does not show a completed generative-AI explanation.

11. User Manual, Deployment and Repository

This chapter gives an end-user workflow for the final web application and a concise developer/deployment guide. The user manual assumes that the account has access to the required organization/project and that external services such as Earth Engine or an AI provider are configured only for features that need them. Figures 13–17 provide visual guidance for project, monitoring, calculation-readiness, review-related, and AI-workspace states; their captions identify the exact state visible in each synthetic demonstration screenshot.

11.1 Sign In and Select Project Context

1. Open the Terra-Audit web application and sign in with an existing account. Use the registration or account-recovery flow when self-service access is enabled.
2. After authentication, open Projects and select an existing project or create a new one if the current role permits it.
3. Use the project/team controls to confirm lead, contributor or viewer membership. UI visibility is not the authorization boundary; the backend rechecks the authenticated user and current role.
4. If a protected action is rejected, confirm the active organization/project context and the user's role rather than changing organization identifiers in the request.

11.2 Register a Field and Build Evidence Context

1. Open the field registration workflow from an authorized project context.
2. Enter the field identity and supported geometry, then select the accounting pathway: rice_awd or cropland_alm_vm0042. The pathway is treated as an immutable routing attribute after registration.
3. Assign the field to the appropriate project/farm membership when required.

4. Open Crop Seasons and create the relevant season. Use the correction workflow for later changes so historical versions remain distinguishable.
5. Add observations, practice events, production records, attachments and other pathway-specific evidence. ALM workflows may additionally require soil sampling plans/strata/samples, laboratory results, custody events, reviewed SOC mapping, livestock schedules and leakage assessment.
6. Resolve validation or review issues rather than substituting a zero/complete value for missing evidence.

11.3 Rice Signal Analytics and Calculation

1. Open Signal Analytics for a rice_awd field and choose the monitoring date window and detector/model option.
2. Start the analysis. Terra-Audit may reuse compatible active/cache data or queue a durable signal_run for the separate worker.
3. Monitor the request until a completed, failed or cancelled state is visible. Earth Engine/worker failures must not be interpreted as a successful analysis.
4. Inspect the resulting flooding/drydown/phenology analytics. These outputs are experimental/supporting evidence; they are not independently validated AWD ground truth.
5. Open Calculations. Where a compatible completed signal job exists, select it as a saved signal source; alternatively enter permitted manual calculation inputs.
6. Review copied source values and any manual overrides, preview the deterministic VM0051 v1.1 calculation, inspect blocking/readiness state, and save a new calculation version.
7. Do not interpret the internal calculated quantity as an issued VCU. External verification/issuance remains outside Terra-Audit.

11.4 ALM Evidence and VM0042 Calculation

1. Use a cropland_alm_vm0042 field and record baseline/project crop seasons and practice schedules.
2. Add production records and any supported livestock schedules.
3. Add the required soil sampling/laboratory/custody evidence and complete the SOC evidence-review/adoption workflow where applicable.
4. Save the supported project/bundle/reporting-period leakage assessment. Missing or unsupported applicable leakage must remain explicit.
5. Open Calculations. The server builds a frozen snapshot of the supported evidence, executes the restricted VMD0054 v1.1 leakage logic and scoped VM0042 v2.2 engine, and evaluates hard gates/readiness.
6. Save the calculation as a new version. A corrected evidence state or later reporting period should create a new calculation rather than rewriting the historical result.
7. Use the snapshot-based ALM calculation export where available for historical audit review.

11.5 Methodology Readiness and Internal Review

1. Open the methodology/readiness area for the relevant project/calculation context.
2. Review the active methodology bundle, corrections, applicability and each requirement status. Blocking, unsupported and reviewer-dependent items are intentionally distinct from the numerical carbon result.

3. A calculation can be stored as a draft even when it is not eligible for internal review.
4. Only a version that reaches `ready_for_review` can enter the normal internal review workflow.
5. Internal reviewers may inspect the version-pinned calculation/evidence context and record assignments, findings, comments and events.
6. Internal review is an application workflow only; it is not external VVB validation or Verra verification.

11.6 AI Explanation

1. From an authorized project context, select an available explanation action such as blocked-calculation explanation, missing evidence, applicability, leakage explanation or calculation-difference explanation.
2. Submit the request. A matching validated cache entry may be reused; otherwise a durable `ai_explain` job is queued.
3. The worker builds/calls the configured provider only after deterministic server context has been prepared. The response is validated for structure, allowed references/citations, numeric support and server-owned actions.
4. Read the resulting explanation as a draft that requires human review. It cannot change readiness, carbon quantities or official evidence state.
5. If the provider is unavailable, the packet is too large or validation fails, use the visible failure state rather than treating missing AI text as a completed determination.

11.7 Export and Troubleshooting

Export — For versioned calculation history, JSON download is supported. Snapshot-backed VM0042 ALM calculation versions additionally support PDF evidence export. Legacy committed verification records retain their separate PDF/JSON/CSV evidence-package exports. For historical work, prefer snapshot-backed outputs where available and observe any legacy-provenance warning.

Signal job remains pending — Check Worker & queue status; a stale or absent worker prevents queued jobs from completing.

Earth Engine failure — Check Earth Engine authentication/project configuration and confirm that the field/date window contains supported Sentinel observations.

Calculation blocked — Inspect readiness and pathway-specific hard gates. Unsupported requirements cannot be bypassed merely to obtain a “ready” label.

AI explanation unavailable — Check provider permission/configuration, worker state and the reported validation/packet error.

Evidence upload fails — Check file size/type and target context; private evidence is downloaded through authenticated application paths.

11.8 Local Installation and Development Setup

Prerequisites include Python 3.12-compatible tooling, Node/npm, and Earth Engine authentication when the SAR pipeline will be used. A minimal local setup is:

1. Clone the public repository and check out the required revision/branch.
2. Create and activate a Python virtual environment.
3. Install backend dependencies with `pip install -r requirements.txt`.

4. For signal analytics, run `earthengine authenticate` and `earthengine set_project <GCP_PROJECT_ID>`.
5. Create a project-root `.env` with at least the required JWT secret, frontend public URL and any external-service configuration. `DATABASE_URL` is optional; when omitted, local SQLite is used.
6. Install frontend dependencies with `cd frontend && npm install`.
7. Start the API with `uvicorn backend.main:app --reload`.
8. Start the durable worker separately with `python -m backend.worker`.
9. Start the frontend with `cd frontend && npm run dev`.
10. Run the final verification commands described in Section 7 before treating a modified revision as report/release-ready.

11.9 Deployment and Public Repository

Public source repository: <https://github.com/knah1d/terra-audit>

Repository screening on 7 October 2026 confirmed the public Terra-Audit repository and its `development` branch. Recorded local verification reports 177 backend/core tests passing, together with passing frontend tests, production build and TypeScript checks; those commands were not rerun during final-report preparation. A successful Vercel deployment status was observed, but the exact revision served by the custom production URL was not independently established. The root README exposes architecture, setup, run and test commands, while configuration examples avoid embedding deployment secrets.

The documented production architecture uses Vercel for the Next.js frontend and Render for the FastAPI backend; an optional Railway deployment configuration remains in the repository. PostgreSQL/Neon-compatible database configuration and local/private-S3-compatible storage are supported, but the exact live backend host, production database, shared artifact storage, worker heartbeat, and deployed revision of the submitted public instance were not independently verified. Hosting is optional for SE801 evaluation and normal development use: the complete application can be run locally with the setup in Section 11.8. Deployment references therefore document supported topology and available evidence without treating unverified production details as confirmed facts.

A traditional desktop installer is not applicable because Terra-Audit is a browser-based web application. Deployment consists of the web frontend, API, database/storage, and separate worker rather than a downloadable executable installer.

12. Evaluation, Limitations and Conclusion

Terra-Audit has progressed from a local single-user prototype into a multi-domain web application for evidence-centered agricultural MRV preparation. The strongest final outcome is the separation of deterministic accounting/readiness logic from assistive satellite, ML and generative-AI components while adding multi-user access, structured evidence, durable jobs,

methodology bundles, calculation versioning and internal review. The recorded software regression results are green, but the project still has clear methodology, scientific, provenance and operational boundaries.

12.1 Final Evaluation

Software architecture — The Next.js/FastAPI/domain/worker structure is materially stronger than the original Streamlit design for multi-user authorization, asynchronous jobs and audit history. Capability-oriented packages keep accounts, projects, evidence, methodology, carbon, signals, jobs, AI, persistence and reporting separate enough that frontend or AI changes do not become calculation authority.

Accounting — The rice and ALM engines are deterministic and regression-tested, with explicit intermediate logic and hard gates. However, both are scoped implementations rather than complete automated implementations of every methodology/program requirement.

Evidence and auditability — Versioned calculation records, evidence snapshots, methodology context, readiness fingerprints and version-pinned internal review provide a substantially stronger audit trail than a mutable spreadsheet/current-state ledger. The remaining weaknesses are primarily incomplete provenance coverage on rice/legacy export paths and application-level rather than database-level immutability.

AI — Classical AWD ML now uses field-grouped evaluation, managed crop models are clearly exploratory, and the generative-AI workflow is constrained by deterministic server packets and validation. These are useful assistive features because they remain outside the authoritative accounting boundary.

Verification — Recorded local results report 177 backend/core tests with no failures or skips, plus passing frontend tests, production build and TypeScript checks. The commands were not rerun during final-report preparation; the recorded result supports software-regression confidence but does not establish scientific, methodology, deployment, or external-verification completeness.

12.2 Methodology and Accounting Limitations

VM0051 v1.1 is implemented only for the scoped QA3/default-factor rice pathway; rice leakage, several fertilizer/burning/monitoring/safeguard elements, and independent evidence that remotely detected events satisfy every methodology requirement remain unsupported or partial. VM0042 v2.2 is likewise scoped to supported QA2-style SOC, uncertainty, selected non-SOC/livestock terms and restricted displacement leakage; QA1/process-model quantification, full DSM/sampling, multi-field leakage, mitigation, nonannual reconciliation and complete sequential-period state remain outside the verified scope.

The methodology registry preserves bundles, corrections, applicability and requirement states, but common.additionality is unsupported, blocking and non-overridable in current behavior. Internal fields such as final_issuance or VCU-style values are therefore internal accounting estimates only; Terra-Audit does not implement Verra registry issuance, and internal review is not VVB verification.

12.3 Scientific Validation Limitations

No independent AWD ground-truth dataset has been completed. Sentinel-1 threshold outputs and RF/XGBoost labels therefore measure model-versus-threshold agreement, not field

accuracy; orbit/incidence-angle/rainfall effects, revisit gaps and field water-level timing remain unvalidated. Future field work should compare detections with water-level tubes, irrigation logs or partner-verified observations and report event precision/recall, timing error and performance across seasons and locations.

12.4 Provenance and Reproducibility Limitations

Versioned calculation snapshots preserve historical inputs/results and ALM PDF exports use frozen calculation context, but there is no database WORM guarantee, cryptographic full-package signature or single canonical provenance artifact covering every Sentinel scene, geometry revision, detector configuration, attachment hash and executable Git revision. Existing fingerprints are subset-specific change detectors rather than a signed audit manifest; legacy paths may still use current/latest context.

12.5 Security and Operational Limitations

Authentication, server-derived organization scope, roles, token revocation, private artifact access and durable job controls provide a useful foundation, but some project-membership enforcement is inconsistent and the database has no general foreign keys or row-level security. Login throttling, malware/content scanning, deletion cleanup, backup/restore, live worker health, exact deployed revision, production database/storage and PostgreSQL migration were not independently verified. No penetration test or production-security certification is claimed.

12.6 Generative-AI Limitations

Generative outputs are validated for structure, allowlisted references, citation membership, numeric support, corrections and server-owned actions/limitations, but this does not prove semantic entailment or factual truth; output remains a human-review draft. The 20-case evaluation runner exists, yet frozen canonical packet fixtures are incomplete, so no provider-quality benchmark is claimed.

12.7 Prioritized Future Work

P0 — Keep the regression suite green while fixing CI branch/type-generation ordering and endpoint authorization; P0 — resolve the additionality/readiness contradiction without weakening the methodology gate; P1 — make rice provenance calculation-specific and add a tamper-evident evidence manifest; P1 — execute independent AWD ground-truth validation in Bangladesh/South Asia; P1 — verify PostgreSQL/shared-storage/worker/backup/deployment recovery and complete the 20-case AI evaluation; P2 — adopt versioned migrations and continue methodology coverage only where authoritative sources and expert/VVB review support it.

12.8 Conclusion

Terra-Audit demonstrates a credible software architecture for evidence-centered agricultural MRV preparation: it organizes multi-user evidence, executes scoped deterministic VM0051/VM0042 calculations, preserves methodology/readiness and versioned history, supports internal review, and provides audit-oriented analytics/exports. It must not be presented as an autonomous verification or issuance platform; independent AWD validation,

complete methodology/program coverage, full provenance sealing, production operational assurance, and external VVB/Verra processes remain separate work.

Appendix A — Selected Technical Persistence Diagram

This appendix retains one implementation-focused persistence view that is especially useful during code review: calculation and internal-review state. The higher-level ERD in Chapter 4 and Table 6 already summarize identity, project/land, monitoring/evidence, soil/production, methodology, jobs, AI and cache domains, so their full technical diagrams are not repeated here.

As in Chapter 4, diagram connectors represent actual keys where declared and otherwise logical/application relationships. The current SQLite design does not generally declare FOREIGN KEY constraints, so the diagram must not be read as claiming database-enforced referential integrity. Snapshot JSON, attachment references and idempotency records are retained here because they are central to calculation and review traceability.

A1(a) Calculations & Internal Review

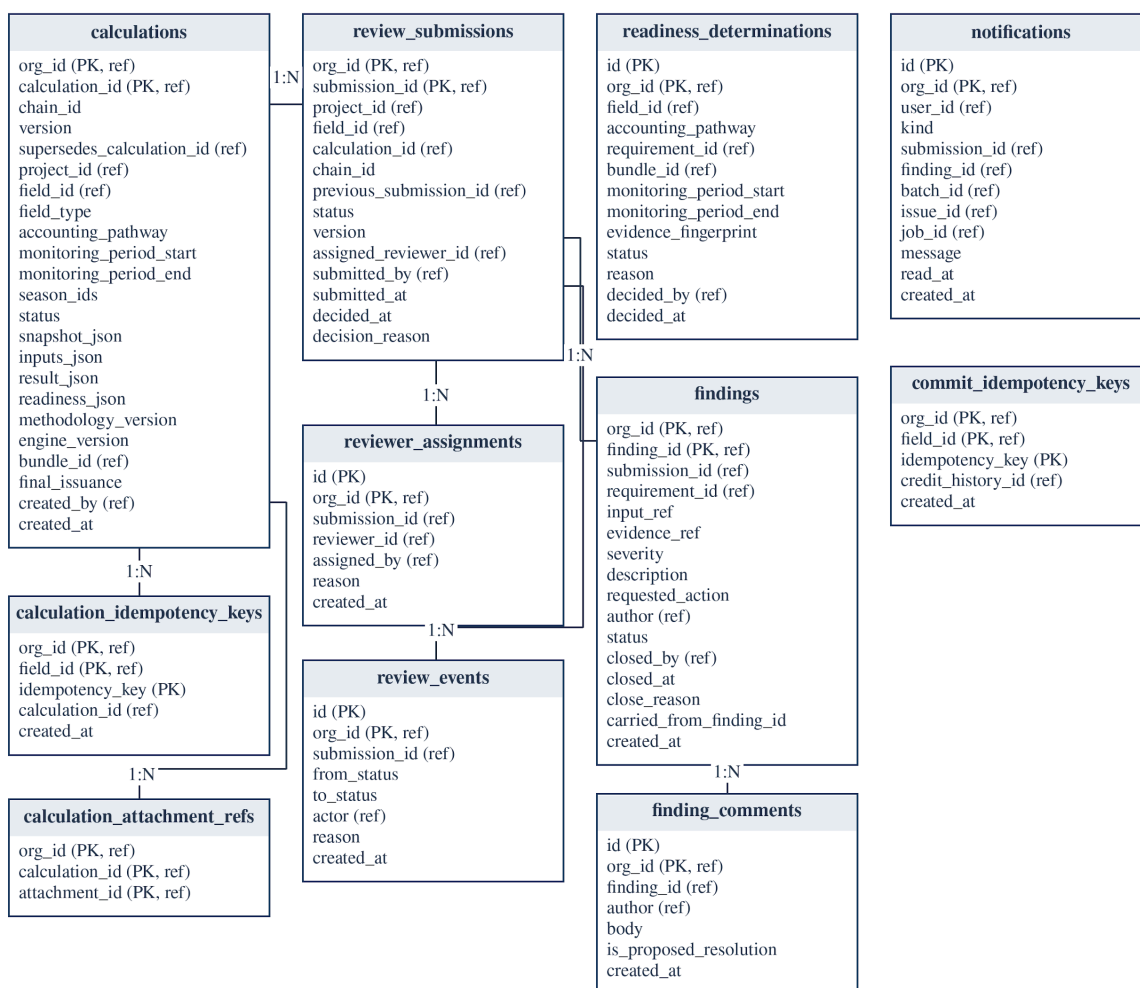


Fig A1(a): Calculation and internal-review persistence tables.

References

1. Verra, VM0051 v1.1 — Methodology for Improved Management in Rice Production Systems. Repository copy: [methodologies/verra/vm0051/VM0051-Improved-Management-in-Rice-Production-Systems-v1.1.pdf](#).
2. Verra, VM0042 v2.2 — Methodology for Improved Agricultural Land Management. Repository copy: [methodologies/verra/vm0042/VM0042v2.2.pdf](#). The repository also contains the June 2026 correction/clarification file [VM0042v2.2_CC_11JUN2026.pdf](#) and VT0014 v1.0 with its correction notice where relevant to the implemented scope.
3. Verra, VMD0054 v1.1 — Methodology for Estimating Leakage from the Displacement of Agricultural Activities. Repository copy: [methodologies/verra/vmd0054/VMD0054-v1.1.pdf](#).
4. IPCC, *2019 Refinement to the 2006 IPCC Guidelines for National Greenhouse Gas Inventories*, Volume 4 (Agriculture, Forestry and Other Land Use), Chapter 5, Table 5.12 (water regime scaling factors) and Table 5.13/5.14 (pre-season and organic-amendment factors); Chapter 10 (livestock enteric fermentation and manure management, cited for the integrated crop-livestock terms in Sections 1.3, 2.1 FR13, and 4.2); Chapter 11, Table 11.1 (N₂O emission factors, including pasture/range/paddock-deposited manure N₂O).
5. IPCC, *Fifth Assessment Report (AR5), Climate Change 2013: The Physical Science Basis* — 100-year Global Warming Potential values (CH₄ = 28, N₂O = 265) used in both carbon estimation engines.
6. Verra, VCS Program Guide; VCS Standard v5.0; and Registration and Issuance Process. Repository copies are stored under [methodologies/verra/standards/](#). These documents are cited only for external program/registration/issuance context; Terra-Audit does not perform registry issuance.
7. European Space Agency / Copernicus Programme, *Sentinel-1 SAR User Guide*. <https://sentinels.copernicus.eu/web/sentinel/user-guides/sentinel-1-sar>
8. Google, *Google Earth Engine API Documentation* (earthengine-api, dataset COPERNICUS/S1_GRD). <https://developers.google.com/earth-engine>
9. R. Lampayan et al., “Adoption and economics of alternate wetting and drying water management for irrigated lowland rice,” *Field Crops Research*, vol. 170, pp. 95–108, 2015 (background on AWD practice).
10. A. Savitzky and M. J. E. Golay, “Smoothing and Differentiation of Data by Simplified Least Squares Procedures,” *Analytical Chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964 (signal smoothing filter used in the data engine).
11. L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
12. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *Proceedings of the 22nd ACM SIGKDD*, pp. 785–794, 2016.